TECHNICAL SPECIFICATION

V4.0 · 2026-04-29

LSIE-MLF

Master Technical Specification — v4.0

Windows Desktop Runtime, Local State, IPC, and Cloud Control Plane

Live Stream Inference Engine — Machine Learning Framework

CONTENT VERSION	SCHEMA VERSION	CLASSIFICATION	HARDENING BASELINE
v4.0	v4.0	CONFIDENTIAL	Gate 0

CONTENTS

Table of contents

0 **Document control and canonical terminology** 3
- 0.1 Document purpose 3
- 0.2 Normative language 3
- 0.3 Canonical terminology registry 3
- 0.4 Version history 5
- 0.5 Specification amendments 6

1 **System philosophy and operational scope** 9

2 **Deterministic system data flow pipeline** 10

3 **Codebase architecture and directory mapping** 12
- 3.1 Directory hierarchy 12
- 3.2 Build context rules 13

4 **Core architecture modules** 14
- 4.A Hardware & transport 14
- 4.B Ground truth ingestion 16
- 4.C Orchestration & synchronization 18
- 4.D Multimodal ML processing 21
- 4.E Experimentation & analytics 23
- 4.F Context enrichment 25

5 **Data governance and security** 26
- 5.1 Ephemeral vault specification 26
- 5.2 Data classification 27

6 **Interface contracts and payload schemas** 28
- 6.1 InferenceHandoffPayload 28
- 6.2 PhysiologicalSampleEvent 35
- 6.3 PhysiologicalContext 37
- 6.4 BanditDecisionSnapshot 39
- 6.5 AttributionEvent 41
- 6.6 OutcomeEvent 45
- 6.7 EventOutcomeLink 46
- 6.8 AttributionScore 47

7 **Mathematical specifications** 49
- 7A AU12 facial action unit geometry 49
- 7B Thompson sampling reward pipeline 52

7C Physiological co-modulation analytics . . . 56
7D Observational acoustic analytics . . . 59
7E Event→Outcome attribution policy . . . 63

8 Deterministic semantic scorer specification 67
8.1 Inference parameter constraints . . . 68
8.2 Deterministic semantic scorer registry . . . 69
8.3 Expected output schema . . . 69
8.4 System prompt (canonical) . . . 70

9 Desktop runtime topology 70
9.1 Desktop process graph . . . 70
9.2 Runtime base . . . 71
9.3 Desktop IPC contract . . . 72

10 Dependency and runtime matrix 76
10.1 System requirements . . . 76
10.2 Python package pinning . . . 77

11 Variable extraction matrix 78

12 Error handling and recovery specification 86
12.1 Hardware device loss . . . 86
12.2 Worker process crash . . . 87
12.3 Temporal drift fault . . . 87
12.4 SharedMemory leak . . . 87
12.5 ML import failure . . . 87
12.6 Pascal speech incompatibility . . . 87
12.7 Acoustic data quality . . . 87
12.8 SQLite Busy/Crash recovery . . . 87
12.9 Reward bounds fault . . . 88
12.10 Network disconnection . . . 88
12.11 Cloud duplicate . . . 88
12.12 Privacy perimeter violation . . . 88
12.13 Operator UI exit . . . 88

13 Autonomous implementation audit checklist 89

SECTION 0

Document control and canonical terminology

0.1 Document purpose

This document is the single authoritative v4.0 specification for the Live Stream Inference Engine — Machine Learning Framework (LSIE-MLF). A developer or autonomous AI agent is required to build the complete v4.0 desktop runtime, local state layer, IPC graph, cloud control plane, and preserved v3.4 mathematical pipeline using only this document. No external documentation is assumed or required.

0.2 Normative language

Throughout this specification, the following terms carry precise normative meaning:

TERM	MEANING
MUST / SHALL / IS REQUIRED TO	The requirement is absolute and non-negotiable.
MUST NOT / SHALL NOT	The prohibition is absolute.
IS DEFINED AS	The term has a single, unambiguous meaning as stated.

0.3 Canonical terminology registry

Every system component has exactly one canonical name. All prior synonyms are retired. Implementations must use only the canonical identifiers listed below.

CANONICAL NAME	RETIRED SYNONYMS	DEFINITION
API Server	`api`, `FastAPI server`, `web server`, `ASGI server`	The FastAPI application process that serves REST endpoints on port 8000. Container name: api.
ML Worker	`worker`, `Celery node`, `Celery worker`, `GPU worker`, `inference worker`	The Celery consumer process that executes all GPU-bound ML inference tasks. Container name: worker.
Message Broker	`Redis`, `queue`, `task queue`, `broker`	The Redis in-memory data store that brokers Celery task queues between the API Server and the ML Worker. Container name: redis.
Persistent Store	`postgres`, `PostgreSQL`, `database`, `relational database`, `DB`	The PostgreSQL relational database that persists analytical metrics and experiment state. Container name: postgres.

CANONICAL NAME	RETIRED SYNONYMS	DEFINITION
Capture Container	`stream_scrcpy, scrcpy container, capture service, stream ingester`	The privileged Docker container responsible for physical USB I/O and raw audio-visual stream extraction. Container name: stream_scrcpy.
IPC Pipe	`FIFO, named pipe, POSIX pipe, audio pipe, kernel pipe, video pipe`	POSIX Named Pipes located at `/tmp/ipc/audio_stream.raw` (audio) and `/tmp/ipc/video_stream.mkv` (video) used for zero-copy data transfer between the Capture Container and the Orchestrator Container.
Ephemeral Vault	`24-hour vault, data vault, transient storage, secure buffer`	The AES-256-GCM encrypted transient storage regime for raw media buffers, subject to mandatory 24-hour secure deletion.
InferenceHandoffPayload	`handoff schema, payload schema, inference payload`	The JSON Schema Draft 07 contract governing data exchange between Module C and Module D, and from Module D to Module E.
Orchestrator Container	`orchestrator, segment producer, Module C worker`	Dedicated Docker container running the Module C orchestration loop. Uses the same image as the ML Worker but with a different CMD (`python3.11 -m services.worker.run_orchestrator`). Produces Celery tasks consumed by the ML Worker.
PhysiologicalChunkEvent	`Physiological Chunk Event, Physiological Sample Event, oura event, HRV event, wearable event, physio event`	A normalized wearable telemetry chunk record produced by the API Server hydration path and enqueued to Redis for the Orchestrator Container to drain. Defined by the PhysiologicalChunkEvent schema in §6.2.
Physiological Context	`physio context, HRV context, biometric context`	A per-subject derived snapshot of physiological state attached to each 30-second InferenceHandoffPayload segment via the _physiological_context dict key. The snapshot may include RMSSD, heart rate, validity_ratio, is_valid, freshness, staleness, source_kind, and derivation_method. Defined by the PhysiologicalContext schema in §6.3.
Physiological State Buffer	`physio buffer, HRV buffer, state cache`	An in-memory rolling buffer in the Orchestrator Container keyed by subject_role ('streamer' or 'operator'). Updated by draining PhysiologicalChunkEvent records from the physio:events Redis list and used to derive scalar physiological snapshots at segment assembly time.
Co-Modulation Index	`co-mod score, synchrony index, physiological correlation`	The Pearson correlation coefficient between 1-minute resampled, time-aligned valid streamer and operator RMSSD sequences over a rolling 10-minute analysis window. Computed by Module E and persisted to the comodulation_log table. Null when fewer than 4 paired observations are available or either aligned series has zero variance.

CANONICAL NAME	RETIRED SYNONYMS	DEFINITION
BanditDecisionSnapshot	`bandit snapshot, decision snapshot, selection snapshot`	The deterministic pre-update Thompson Sampling decision record captured by Module C at arm-selection time and attached to InferenceHandoffPayload segments for downstream attribution and offline evaluation.
AttributionEvent	`attribution event, event ledger row, encounter attribution record`	The canonical derived record persisted by Module E representing a stimulus-linked interaction event eligible for downstream event→outcome attribution under §7E.
OutcomeEvent	`conversion event, terminal event, outcome row`	The canonical derived record persisted by Module E representing a delayed downstream outcome such as creator_follow for event→outcome attribution under §7E.
EventOutcomeLink	`attribution link, event link, causal link row`	The canonical derived record persisted by Module E representing deterministic eligibility linkage between an AttributionEvent and an OutcomeEvent under a versioned horizon/linking rule.
AttributionScore	`attribution metric, score row, ledger score`	The canonical derived record persisted by Module E representing a method-specific attribution or observational score attached to an event or event→outcome pair.
Desktop Process Graph	`['Docker Compose runtime topology', 'container graph']`	The v4.0 six-process Windows-native runtime boundary for UI/API, capture, orchestration, GPU ML, analytics state, and cloud sync.
SharedMemory Audio Segment	`['base64 _audio_data JSON transport', 'audio IPC pipe payload']`	A 30-second PCM window staged in multiprocessing.shared_memory.SharedMemory and referenced by Queue control metadata.
Gate 0 Replay Corpus	`['pre-sprint fixture set']`	The golden conformance corpus that proves v4.0 transport changes preserve v3.4 deterministic outputs.
Cloud Control Plane	`['remote PostgreSQL direct access']`	The HTTPS/OAuth/PKCE service boundary for experiment bundles, telemetry ingest, posterior delta upload, and session management.

0.4 Version history

VERSION	DATE	DESCRIPTION
v1.0	2026-03-12	Initial unified specification.
v2.0	2026-03-12	Production-hardened. Applied 15-rule deterministic audit: canonical terminology, module contracts, data-flow pipeline, dependency pinning, AU12 Python fix, IPC lifecycle, temporal sync formulas, Ephemeral Vault hardening, Docker topology expansion, LLM determinism constraints, error-handling matrices, variable matrix expansion, and autonomous implementation audit.
v3.0	2026-04-01	Mathematical recipe (Thompson Sampling, continuous reward, fractional Beta-Bernoulli update), E2E gap fixes, hardening baseline, 6 spec amendments incorporated (cuDNN 8, Ubuntu 24.04, orchestrator container, scrcpy v3.3.4, audio chunk 1/30s, INT8 locked), 63 new tests, reward pipeline formalization.

VERSION	DATE	DESCRIPTION
v3.1	2026-04-15	Physiological co-modulation extension. Added Oura Ring webhook ingestion (§4.B.2), Orchestrator physiological state buffer (§4.C.4), Module E co-modulation analytics (§4.E.2), PhysiologicalChunkEvent and PhysiologicalContext schemas (§6.2, §6.3), RMSSD co-modulation math (§7C), physiological data governance, failure matrix entries, variable matrix entries, and audit checklist items. §7B Thompson Sampling reward pipeline explicitly unchanged.
v3.2	2026-04-17	Physiological ingestion model corrected. Oura webhook handling no longer assumes provider-delivered scalar 5-minute RMSSD payloads. Webhooks are treated as change notifications that trigger OAuth2-backed hydration, normalized physiological chunk transport, rolling RMSSD derivation in Module C, validity gating, and 1-minute co-modulation resampling. Operator dashboard stack migrated from Streamlit to dual PySide6 host surfaces: a retained single-file developer debug tool and a modular production operator console. §7B reward pipeline remains unchanged.
v3.3	2026-04-23	Observational acoustic analytics extension. Added stimulus-windowed acoustic analysis in Module D, new mathematical recipe (§7D), canonical Module D → Module E payload fields, Persistent Store schema-oriented acoustic field additions, deterministic acoustic data-quality handling, and new audit checklist items. §7B Thompson Sampling reward pipeline remains unchanged.
v3.4	2026-04-24	Attribution analytics extension. Added Module E Attribution Analytics (§4.E.3), stable segment identity and BanditDecisionSnapshot carriage in InferenceHandoffPayload (§6.1), attribution schemas (§6.4), Event→Outcome Attribution Policy (§7E), deterministic semantic scorer registry with local cross_encoder primary and optional llm_gray_band fallback (§8), bounded semantic reason codes, variable matrix additions, and new audit controls. §7B Thompson Sampling reward pipeline remains unchanged.
v4.0	2026-04-29	Desktop runtime pivot. Replaces Docker Compose runtime with Windows Tier 1 six-process desktop graph; introduces python-build-standalone plus uv runtime hydration, Queue plus SharedMemory IPC, SQLite WAL local state, cloud control plane, deterministic offline arm selection, posterior delta sync, Turing production GPU floor, Pascal CPU-speech developer override, and Gate 0 replay invariants. §7B reward mathematics remain unchanged.

0.5 Specification amendments

AMENDMENT	REF	ACCEPTED CHANGE
`SPEC-AMEND-001` 2026-03-20	9.1	Original: Worker image: `nvidia/cuda:12.2.2-cudnn9-runtime-ubuntu22.04` Amended: Worker image: `nvidia/cuda:12.2.2-cudnn8-runtime-ubuntu22.04` Rationale: cuDNN 9 lacks SM 6.1 (Pascal) binaries. Target hardware (GTX 1080 Ti) requires cuDNN 8 for dp4a INT8 vectorization. Incorporated into v3.0 spec text.

AMENDMENT	REF	ACCEPTED CHANGE
SPEC-AMEND-002 2026-03-20	9.1	Original: Capture Container image: `ubuntu:22.04` Amended: Capture Container image: `ubuntu:24.04` Rationale: scrcpy v3.1+ prebuilt Linux x86_64 binaries require GLIBC 2.38+, unavailable in Ubuntu 22.04 (ships GLIBC 2.35). Ubuntu 24.04 ships GLIBC 2.39. Incorporated into v3.0 spec text.
SPEC-AMEND-003 2026-03-20	9.1	Original: 5 containers (redis, postgres, stream_scrcpy, worker, api). Module C runs inside the worker process. Amended: 6 containers. Orchestrator runs as a separate container using the same image as the worker but with a different CMD (`python3.11 -m services.worker.run_orchestrator`). Rationale: The orchestrator is the PRODUCER of Celery tasks while the worker is the CONSUMER. Separating them prevents the Celery consumer's concurrency model from conflicting with the orchestrator's asyncio event loop and persistent FFmpeg/PyAV subprocesses. Incorporated into v3.0 spec text.
SPEC-AMEND-004 2026-03-20	4.A.1	Original: scrcpy v2.x with single-instance multiplexed audio/video capture. Amended: scrcpy v3.3.4 with dual-instance architecture: audio on port range 27100:27199 and video on port range 27200:27299. Direct `--record` to pipe replaces dd/fd3 for video path. Rationale: Dual-instance design eliminates cross-stream coupling and allows either scrcpy process to be restarted independently. Direct `--record` to MKV streaming container avoids fragile dd byte piping. Incorporated into v3.0 spec text.
SPEC-AMEND-005 2026-03-20	4.C.3	Original: Audio chunk size: 1 second (32000 bytes). Amended: Audio chunk size: 1/30 second (~1067 bytes) via `int(bytes_per_second / 30)`. Rationale: Aligns audio consumption cadence with 30 FPS video frame rate for temporal synchronization. Incorporated into v3.0 spec text.
SPEC-AMEND-006 2026-03-20	4.D.1	Original: TranscriptionEngine `compute_type` configurable at deployment time. Amended: TranscriptionEngine fixes class constant `_COMPUTE_TYPE` to `"int8"`. Not exposed as a deployment-time option. Rationale: Downstream effect of SPEC-AMEND-001: INT8 uses dp4a which is available on Pascal SM 6.1, FP16 is not. Locking the value prevents accidental misconfiguration. Incorporated into v3.0 spec text.

AMENDMENT	REF	ACCEPTED CHANGE
SPEC-AMEND-007 2026-04-15	§4.B.2, §4.C.4, §4.E.2, §6.2, §6.3, §7C, §5, §11, §12, §13	Original: No physiological telemetry ingestion, no wearable data integration, no co-modulation analytics. Amended: v3.1 adds physiological co-modulation extension: Oura Ring webhook ingestion via API Server (§4.B.2), Physiological State Buffer in Orchestrator (§4.C.4), optional _physiological_context on InferenceHandoffPayload segments, Module E co-modulation analytics (§4.E.2), RMSSD Pearson correlation math (§7C), physiology_log and comodulation_log database tables, and associated data governance, failure handling, and audit items. §7B reward pipeline unchanged. Rationale: Behavioral markers (AU12) are subject to performative masking. Integrating autonomic nervous system data (RMSSD via Oura Ring) provides a non-deceptive physiological signal for evaluating operator-streamer interaction quality. Deployed as observational analytics in v3.1; reward integration deferred to a future revision.
SPEC-AMEND-008 2026-04-17	§4.E.1, §10.2	Original: Operational metrics and experiment state are visualized through a Streamlit dashboard. Dependencies list includes Streamlit. Amended: Streamlit dashboard retired. Operator visualization is delivered through two PySide6 surfaces running on the operator host: (a) scripts/debug_studio.py retained as a single-file developer diagnostic GUI; (b) services/operator_console/ implemented as a modular production operator dashboard. All operator-surface data access flows through the FastAPI server. No direct Postgres or Redis coupling is added to the operator host. requirements/worker.txt removes streamlit. requirements/cli.txt pins PySide6. Rationale: Streamlit's rerun-oriented web model is not suitable for stateful operator workflows that require live video and metric readback, device-level operator actions, and long-running native sessions. PySide6 provides persistent, native host-side operator surfaces while keeping developer diagnostics separate from the production console.
SPEC-AMEND-009 2026-04-17	§4.B.2, §4.C.4, §6.2, §6.3, §11.3, §7C	Original: Oura webhook path assumes the provider delivers aggregated 5-minute RMSSD and heart-rate values directly into the webhook transport, and Module C stores only the latest scalar snapshot. Amended: Physiological ingestion model revised. Oura Ring webhook deliveries are treated as change notifications rather than authoritative scalar RMSSD samples. The API Server verifies, deduplicates, subject-maps, and enqueues hydration work. OAuth2-backed hydration fetches provider resources and normalizes them into PhysiologicalChunkEvent records for Orchestrator consumption. Module C maintains rolling per-subject physiological buffers, derives scalar RMSSD snapshots over a trailing 5-minute window, computes validity_ratio, applies a strict validity gate, and attaches derived _physiological_context snapshots to segment payloads. Module E computes co-modulation on 1-minute resampled, time-aligned valid RMSSD snapshots over a 10-minute window. §7B reward pipeline unchanged. Rationale: The prior scalar-webhook assumption is structurally incorrect for daytime Oura ingestion. The corrected design preserves the existing API → Redis → Orchestrator boundary, keeps physiological persistence scalar-only, and makes the co-modulation math internally consistent with the 10-minute analysis window.

AMENDMENT	REF	ACCEPTED CHANGE
SPEC-AMEND-010 2026-04-24	§4.C.2, §4.E.3, §6.1, §6.4, §7E, §8, §11, §13	Original: No first-class attribution ledger, no stable segment_id in InferenceHandoffPayload, semantic evaluation specified only as deterministic Azure OpenAI free-form reasoning output, and no event→outcome attribution schemas. Amended: v3.4 adds attribution analytics as an observational extension: stable segment_id and BanditDecisionSnapshot in InferenceHandoffPayload; Module E Attribution Analytics (§4.E.3); AttributionEvent, OutcomeEvent, EventOutcomeLink, AttributionScore, and BanditDecisionSnapshot schemas (§6.4); Event→Outcome Attribution Policy (§7E); deterministic semantic scorer registry with local cross_encoder primary and optional llm_gray_band fallback; bounded semantic reason codes; and new attribution audit controls. §7B reward pipeline unchanged. Rationale: The system requires first-class event identity, future off-policy-evaluation-enabling decision capture, and a lower-latency deterministic semantic chokepoint improvement without violating the audited reward path.

SECTION 1

System philosophy and operational scope

LSIE-MLF v4.0 ingests USB-tethered multimodal streams—PCM audio, H.264 video, and chat metadata—inside a Windows-native desktop process graph and runs concurrent ML pipelines to produce structured metrics. Thompson Sampling continues to use AU12 smile intensity as a dense proxy reward for the sparse creator-follow event.

The architecture remains schema-first and deterministic. USB media transport, temporal synchronization, message passing, local state, cloud sync, and ML inference must behave predictably across independent audio, video, chat, physiological, and cloud clocks.

The v4.0 runtime replaces Docker Compose, Redis, Celery, and local PostgreSQL as normative desktop dependencies with six Windows processes, multiprocessing.Queue control messages, SharedMemory audio transport, SQLite WAL local state, and HTTPS cloud synchronization.

The system applies strict data minimization. Raw biometric media is processed transiently and must not persist locally or in cloud storage; persistent state is limited to derived, normalized outputs under the desktop privacy and cloud perimeter rules in §5.

SECTION 2

Deterministic system data flow pipeline

This section defines the complete end-to-end v4.0 data path through the Windows-native LSIE-MLF desktop runtime. Every transition specifies the process boundary, transport mechanism, payload format, synchronization mechanism, and error behavior. The preserved v3.4 analytical contracts remain invariant across the desktop transport.

STAGE 1 · MOBILE HARDWARE → CAPTURE_SUPERVISOR

TRANSPORT	USB tether via host ADB and scrcpy v3.3.4 supervised by Win32 Job Objects. Audio and video scrcpy instances use dual-port ranges: audio 27100:27199 and video 27200:27299.
PAYLOAD	Audio: raw PCM signed 16-bit little-endian (s16le), 48 kHz mono over an in-memory FFmpeg pipe. Video: H.264 carried in an MKV streaming container read by the local video consumer; decoded frames never cross a process boundary.
SYNC	Android hardware clock polled through adb shell every 30 seconds. Drift offsets are emitted to the desktop Queue and later applied by module_c_orchestrator.
ERROR BEHAVIOR	If USB is lost, capture_supervisor logs the event, terminates supervised native children through their Job Object, polls reconnection every 2 seconds for 60 seconds, and either revives the capture graph or requests graceful session shutdown.

STAGE 2 · CAPTURE_SUPERVISOR → MODULE_C_ORCHESTRATOR

TRANSPORT	multiprocessing.Queue for control, in-memory FFmpeg pipes for media handoff inside the local process tree.
PAYLOAD	CaptureStatus, DriftOffset, NativeProcessHealth, and pipe endpoint descriptors. No persistent raw media files are emitted.
SYNC	corrected_ts = original_ts + drift_offset. DRIFT_FREEZE_AFTER_FAILURES remains 3 and DRIFT_RESET_TIMEOUT remains 300 seconds.
ERROR BEHAVIOR	After three consecutive drift poll failures the last valid offset freezes; after 300 seconds the drift state resets. FFmpeg restart logic and video-pipe revival preserve v3.4 behavior while changing the supervising surface from container to process.

STAGE 3 · MODULE_C_ORCHESTRATOR INTERNAL MEDIA SYNCHRONIZATION

TRANSPORT	In-process deques and MediaPipe Face Mesh execution. Video frames remain in process memory and are not serialized.
PAYLOAD	AU12 observation series, segment window metadata, physiological context snapshot, and stable segment identity.
SYNC	SEGMENT_WINDOW_SECONDS = 30. segment_id = SHA256(session_id || canonical window_start || canonical window_end).
ERROR BEHAVIOR	If AU12 or physiological context is unavailable, downstream observational fields become deterministic nulls/false validity flags without blocking reward-path persistence.

STAGE 4 · MODULE B / CLOUD EXPERIMENT CACHE → MODULE_C_ORCHESTRATOR

TRANSPORT	Local Queue events and read-only SQLite experiment mirror populated by signed ExperimentBundle sync.
PAYLOAD	Ground-truth chat events, expected greeting configuration, experiment metadata, and signed active-arm policy bundle references.
SYNC	Events are normalized to UTC, then aligned to corrected media time. Offline arm selection uses deterministic seeded Thompson Sampling when cloud is unavailable.
ERROR BEHAVIOR	Cloud unavailability does not block local inference; cached signed bundles remain active until expiry policy, and pending posterior deltas queue locally.

STAGE 5 · MODULE_C_ORCHESTRATOR → GPU_ML_WORKER

TRANSPORT	multiprocessing.Queue control envelope plus multiprocessing.shared_-memory.SharedMemory for each 30-second PCM window.
PAYLOAD	Queue control message includes segment_id, shared_memory_name, byte_length, checksum, _au12_series, _stimulus_time, _active_arm, _experiment_id, _expected_greeting, _x_max, _bandit_decision_snapshot, and optional _physiological_context. It must not include serialized _audio_data on the desktop path.
SYNC	The control envelope wraps the canonical InferenceHandoffPayload. The desktop path changes only audio transport, not analytical field semantics.
ERROR BEHAVIOR	SharedMemory producers register lifecycle ownership; gpu_ml_worker consumers attach with track=False. On acknowledgement, the owner zeroizes, closes, and unlinks the segment. Dirty State Recovery unlinks orphaned lsie_ipc_* segments at startup.

STAGE 6 · GPU_ML_WORKER → ANALYTICS_STATE_WORKER

TRANSPORT	multiprocessing.Queue result messages carrying bounded, schema-validated derived outputs.
PAYLOAD	Transcript, semantic match result, bounded reason code, acoustic metrics, optional shadow scores, and references to preserved handoff metadata.
SYNC	Result timestamps remain aligned to the segment window and stimulus time. Raw PCM is never returned or persisted.
ERROR BEHAVIOR	Acoustic insufficiency, semantic scorer ambiguity, or transcription failure produces deterministic validity flags and null dependent outputs without crashing the worker or blocking state persistence.

STAGE 7 · ANALYTICS_STATE_WORKER → SQLITE WAL AND CLOUD_SYNC_WORKER

TRANSPORT	Single-writer SQLite transactions plus local Queue messages for pending HTTPS upload.
PAYLOAD	RewardResult, posterior deltas, metrics, physiological summaries, acoustic summaries, attribution events, and upload queue rows. No raw media, raw provider bodies, complete PhysiologicalChunkEvent payloads, or free-form semantic rationales.
SYNC	analytics_state_worker computes §7B reward invariants and emits posterior deltas exactly once per segment/arm/client idempotency key.
ERROR BEHAVIOR	SQLite busy timeout is 5000 ms. If cloud sync fails, rows remain in pending_uploads with retry/backoff; local posterior state remains authoritative for offline decisions.

STAGE 8 · CLOUD_SYNC_WORKER ↔ CLOUD CONTROL PLANE

TRANSPORT	HTTPS with OAuth 2.0 Authorization Code + PKCE. Refresh tokens are stored in Windows Credential Manager through keyring.
PAYLOAD	Signed ExperimentBundle downloads, segment telemetry uploads, posterior_delta uploads, and session lifecycle messages validated with extra=forbid schemas.
SYNC	Posterior deltas apply with INSERT … ON CONFLICT DO NOTHING and atomic posterior UPDATE only on insert success.
ERROR BEHAVIOR	Network failure degrades to offline cache mode. The privacy perimeter rejects raw PCM, decoded frames, voiceprints, raw provider hydration bodies, complete physiological chunks, and free-form semantic rationales.

SECTION 3

Codebase architecture and directory mapping

The repository remains a segmented monorepo, but v4.0 replaces the normative Docker Compose runtime with a Windows-native desktop application surface.

The six-process graph isolates UI/API, capture supervision, orchestration, GPU ML imports, analytics state writes, and cloud synchronization.

Only gpu_ml_worker may import torch, ctranslate2, mediapipe, and faster_whisper as a combined heavy-ML process boundary; module_c_orchestrator may import MediaPipe only for AU12/Face Mesh where required.

3.1 Directory hierarchy

PATH	PURPOSE
/services/desktop_launcher/	Windows launcher shell. Downloads and hash-verifies python-build-standalone, runs uv sync –frozen, manages repair/reinstall, and writes the install manifest.
/services/desktop_app/processes/ui_api_shell.py	Hosts the PySide6 operator console and localhost FastAPI shell.
/services/desktop_app/processes/capture_supervisor.py	Supervises ADB, scrcpy, and FFmpeg native subprocesses through Win32 Job Objects and emits capture/drift events.
/services/desktop_app/processes/module_c_-orchestrator.py	Owns media synchronization, drift application, AU12 extraction, physiological context buffering, segment assembly, and desktop IPC production.
/services/desktop_app/processes/gpu_ml_worker.py	Only process allowed to import the complete torch, ctranslate2, mediapipe, faster_whisper ML dependency set.
/services/desktop_app/processes/analytics_state_worker.py	Sole SQLite writer. Computes §7B reward, posterior updates, attribution outputs, and posterior delta records.
/services/desktop_app/processes/cloud_sync_worker.py	Owns HTTPS sync, OAuth/PKCE token refresh, signed ExperimentBundle download, telemetry upload, and posterior delta upload.
/services/desktop_app/ipc/	Queue message models, SharedMemory segment lifecycle helpers, checksums, zeroization, and Dirty State Recovery sweep.
/services/desktop_app/state/sqlite_schema.py	SQLite WAL DDL mirror of derived local state tables; JSONB fields from cloud DDL are stored as TEXT with Pydantic validation.
/services/cloud_api/	Cloud control plane API for telemetry ingest, experiment bundles, sessions, OAuth token exchange, and posterior delta application.
/packages/ml_core/	Shared mathematical and inference utilities.
/packages/schemas/	Pydantic and JSON Schema contracts for handoff, attribution, physiology, desktop IPC, cloud telemetry, and posterior deltas.
/requirements/v4_desktop_lock/	uv-managed desktop dependency lock and lock-check scripts.
/tests/fixtures/v4_gate0/	Golden replay corpus containing captured media/event/drift inputs and expected schema-validated outputs.

3.2 Build context rules

v4.0 builds a signed Windows desktop launcher and hydrates the ML runtime with python-build-standalone plus uv.lock. PyInstaller one-folder mode may wrap the launcher shell only; heavy ML wheels must not be re-extracted on every app launch.

SECTION 4

Core architecture modules

The modules defined below form the operational spine of LSIE-MLF v4.0. The v3.4 module identities and analytical contracts survive, but the runtime boundary changes from container dispatch to a Windows-native six-process desktop graph. Each module declares its process owner.

4.A Hardware & transport

Purpose. Extract low-latency audio and video streams from tethered mobile hardware and transport them into the containerized environment without REST or broker serialization overhead.

Technology stack. scrcpy v3.3.4, ADB, FFmpeg pipes, Win32 Job Objects, dual scrcpy port ranges 27100:27199 and 27200:27299, no persistent raw media staging

4.A.1 EXECUTION DETAILS

Module A maps the host `/dev/bus/usb` subsystem into the Capture Container and starts two coordinated scrcpy v3.3.4 processes against the same Android device. One process captures audio and the other captures video.

The dual-instance design keeps the transport path simple and restartable. Audio is written directly to `/tmp/ipc/audio_stream.raw` through scrcpy's `--record` path, while video is written directly to `/tmp/ipc/video_stream.mkv`. Splitting the streams avoids cross-stream coupling and allows either scrcpy process to be relaunched independently when only one media path fails.

The capture design uses intentionally asymmetric pipe lifecycle handling so audio continuity and video container recovery each fail in the intended way under independent scrcpy restarts.

Startup is staggered by 4 seconds to prevent concurrent ADB server pushes to the device. Before every capture cycle, the container clears `adb forward` and `adb reverse` tunnels, then launches the audio instance first and the video instance second. Both instances use `--no-playback` and write into the shared ipc-share volume instead of routing media through REST endpoints or Redis payloads.

- Audio scrcpy flags: `--no-video`, `--no-playback`, `--audio-codec=raw`, `--audio-buffer=30`, `--audio-dup`, `--record=$AUDIO_PIPE`, `--port=27100:27199`, `--tunnel-host=$HOST_IP`.
- Video scrcpy flags: `--no-audio`, `--no-playback`, `--video-codec=h264`, `--max-fps=30`, `--record=$VIDEO_PIPE`, `--record-format=mkv`, `--port=27200:27299`, `--tunnel-host=$HOST_IP`.

> **VIDEO FIFO INTENTIONALLY LEFT UNSHIELDED**
>
> Do not symmetrize the pipe handling. The shell keeps only the audio FIFO open with `exec 3<> $AUDIO_PIPE`; the video FIFO stays unshielded so a PyAV-side failure breaks the write path, forces video scrcpy to exit, and lets the next launch emit a fresh MKV header.

IPC PIPE LIFECYCLE SPECIFICATION

1. **Directory creation:** The Capture Container initializes the shared IPC mount by running `mkdir -p /tmp/ipc` before any scrcpy process is started.
2. **Pipe cleanup and creation:** `setup_pipes()` removes stale FIFO files with `rm -f` and recreates both `/tmp/ipc/audio_stream.raw` and `/tmp/ipc/video_stream.mkv` with `mkfifo`.
3. **Audio fd 3 shield:** The shell opens `exec 3<> $AUDIO_PIPE` so the audio pipe stays open across reader or scrcpy restarts and the writer is decoupled from consumer timing.
4. **Video pipe left unshielded:** No extra descriptor is opened for `/tmp/ipc/video_stream.mkv`; if the PyAV consumer drops the pipe, the video scrcpy process exits and the next launch emits a fresh MKV header.
5. **Device wait gate:** `wait_for_device()` polls `adb devices` every 2 seconds for up to 60 seconds and terminates the container if no Android device reports a ready device state.
6. **ADB tunnel cleanup:** Each capture loop starts by running `adb forward --remove-all` and `adb reverse --remove-all` so stale tunnels cannot collide with the audio and video port ranges.
7. **Audio stream launch:** The audio scrcpy instance launches first and writes directly to `/tmp/ipc/audio_stream.raw` with `--record` while binding port range 27100:27199 through `$HOST_IP`.
8. **Staggered video launch:** After a 4-second delay, the video scrcpy instance launches and writes H.264 in an MKV streaming container directly to `/tmp/ipc/video_stream.mkv` on port range 27200:27299.
9. **Monitoring and restart:** The container waits on both scrcpy PIDs with `wait -n`; when either process exits, both children are cleaned up, the device is rechecked, and the dual-stream capture loop restarts.
10. **Cross-launch cleanup:** On the next container start, `setup_pipes()` again runs `rm -f` on both FIFO paths before recreating them, preventing stale shared-volume pipe files from surviving across launches.

INPUTS	USB-connected Android device producing raw PCM audio (s16le 48 kHz) and H.264 encoded video via scrcpy.
OUTPUTS	Continuous PCM audio stream (s16le, 48 kHz, mono) written to `/tmp/ipc/audio_stream.raw`, and continuous H.264 video in an MKV streaming container written to `/tmp/ipc/video_stream.mkv`; both are exposed as IPC Pipes on the shared Docker volume ipc-share.
DEPENDENCIES	scrcpy v3.3.4, ADB server via `ADB_SERVER_SOCKET=tcp:host.docker.internal:5037`, `/dev/bus/usb` device passthrough, Docker volume ipc-share mounted at `/tmp/ipc/` , environment variables `SDL_VIDEODRIVER=dummy`, `XDG_RUNTIME_DIR=/tmp`, `SCRCPY_SERVER_PATH`, and `SCRCPY_ICON_PATH`, plus dedicated scrcpy port ranges 27100:27199 (audio) and 27200:27299 (video).
SIDE EFFECTS	Creates two named pipes (`/tmp/ipc/audio_stream.raw` and `/tmp/ipc/video_stream.mkv`), opens file descriptor 3 as an audio-pipe shield, spawns persistent audio and video scrcpy processes, and establishes ADB forward/reverse tunnels for port ranges 27100:27199 and 27200:27299.
FAILURE MODES	1. USB disconnect causes wait_for_device() to poll every 2 seconds for up to 60 seconds before the container exits. 2. If the audio scrcpy process crashes, wait -n detects the exit, both scrcpy PIDs are cleaned up, and the loop restarts while fd 3 keeps /tmp/ipc/audio_stream.raw open. 3. If the video path breaks, the unshielded /tmp/ipc/video_stream.mkv pipe lets the video scrcpy process fail so the next launch writes a fresh MKV header for PyAV. 4. Each capture cycle removes adb forward/reverse tunnels to prevent port collisions, and if the downstream consumer lags, pipe writes may return EAGAIN so overflowed data is silently discarded per §12.

4.B Ground truth ingestion

Purpose. Ingest real-time contextual metadata, chat streams, and user interactions to create ground-truth datasets.

Technology stack. Python 3.11.x, TikTokLive connector, signed ExperimentBundle cache, localhost FastAPI shell, UTC event normalization

4.B.1 EXECUTION DETAILS

The system continuously ingests live chat and interaction events from the platform using the TikTokLive Python connector.

Connection requires ByteDance anti-crawler signatures including `X-Bogus` and `msToken`. Signature generation is delegated to the EulerStream signature server.

Once connected the ingestion module enforces the `Action_Combo` constraint where multiple events must coincide temporally to form a valid ground truth signal.

Incoming protobuf messages are parsed into Python dictionaries and high-value composite events are staged for later synchronization with inference outputs.

4.B.2 PHYSIOLOGICAL INGESTION ADAPTER

Physiological webhook ingestion is implemented as a FastAPI POST route in the API Server container. It is separate from TikTok WebSocket ingestion and has its own authentication, validation, and transport rules.

Oura webhook deliveries trigger an OAuth2-backed hydration step that fetches the updated provider resource and normalizes it into physiological chunk records.

- Verify `x-oura-signature` with `OURA_WEBHOOK_SECRET`, resolve `subject_role`, and suppress duplicates with Redis `SETNX physio:seen:{unique_id}`.
- Parse only the notification metadata needed to route hydration.
- Enqueue hydration work on `physio:hydrate`; the hydrator refreshes tokens as needed and emits normalized PhysiologicalChunkEvent records using `source_kind` ibi or session.
- Append normalized chunk events to `physio:events` with `RPUSH` for Orchestrator consumption.
- The route emits explicit HTTP failures for signature, schema, and Redis-availability problems.

OURA WEBHOOKS ARE CHANGE NOTIFICATIONS, NOT RMSSD SAMPLES

Treat Oura Ring webhooks as change notifications only. Do not read them as authoritative `rmssd_ms` samples or derive RMSSD from generic heartrate notifications; verify, deduplicate, hydrate the provider resource, and normalize the result into PhysiologicalChunkEvent records.

RAW WEBHOOK/PROVIDER PAYLOADS MUST NEVER PERSIST

Keep raw webhook JSON and hydrated provider response bodies transient. They must not be persisted; only normalized downstream records may continue, with `401` on signature failure, `422` on schema failure, and `503` when Redis is unavailable.

INPUTS	Authenticated TikTok Webcast WebSocket event stream using EulerStream-signed URL parameters. Additionally, authenticated physiological webhook payloads arrive via the API Server and are enqueued to a Redis list for downstream consumption (§4.B.2).
OUTPUTS	Parsed dictionaries representing high-value TikTok events containing `uniqueId`, `event_type`, `timestamp_utc`, `gift_value`, and `combo_events`; normalized PhysiologicalChunkEvent records appended to Redis list `physio:events`; and hydration work items queued on `physio:hydrate`.
DEPENDENCIES	TikTokLive library, EulerStream signature API, network access to TikTok Webcast servers, FastAPI route handling, Redis client, Oura webhook secret, Oura OAuth2 hydration client and token storage, and schema validation for PhysiologicalChunkEvent.
SIDE EFFECTS	Maintains persistent WebSocket connection and refreshes signature tokens during reconnects. Verifies physiological webhook signatures, deduplicates provider notifications, enqueues hydration work, and appends normalized physiological chunk records to Redis.
FAILURE MODES	1. WebSocket disconnection triggers exponential backoff reconnect. 2. EulerStream outage results in degraded mode. 3. Malformed protobuf events are logged and discarded. 4. Invalid physiological webhook signatures are rejected with 401. 5. Malformed physiological payloads are rejected with 422. 6. Redis unavailability returns 503 for provider retry. 7. OAuth2 hydration failures are retried asynchronously without fabricating physiological defaults.

4.C Orchestration & synchronization

Purpose. Module C runs as a dedicated Orchestrator Container built from the worker image and started with `python3.11 -m services.worker.run_orchestrator`. It polls ADB for drift correction, resamples 48 kHz PCM to 16 kHz through a persistent FFmpeg subprocess, processes video frames for AU12, assembles fixed 30-second segments, and dispatches InferenceHandoffPayload tasks to the Celery worker.

Technology stack. asyncio, multiprocessing.Queue producer, SharedMemory producer, PyAV-backed local video consumer, MediaPipe Face Mesh, ADB drift offset application, collections.deque buffering

4.C.1 TEMPORAL DRIFT CORRECTION

DriftCorrector polls the Android epoch every 30 seconds by invoking ADB and computes the drift offset as host UTC minus Android epoch before segment timestamps are emitted.

Each corrected timestamp is produced by adding the cached drift offset to the original timestamp, and the orchestrator warns when absolute drift exceeds 150 milliseconds.

When ADB polling degrades, drift correction falls back to deterministic cached-offset behavior instead of introducing unstable timestamp jitter.

DRIFT CORRECTION FREEZES AFTER REPEATED ADB FAILURES

After three consecutive ADB poll failures, DriftCorrector freezes the last cached offset and keeps returning it while frozen. If the frozen state lasts 300 seconds, reset the offset to zero until a later successful poll refreshes it.

4.C.2 VIDEO TELEMETRY AND SEGMENT ASSEMBLY

The Orchestrator Container keeps a long-lived video capture thread on `/tmp/ipc/video_stream.mkv`, samples the latest frame inside the asyncio loop, and lazy-loads MediaPipe Face Mesh together with the AU12 normalizer on first use.

Before stimulus injection, incoming frames only accumulate the *B_neutral* calibration baseline. After the greeting trigger is recorded, the orchestrator stores a drift-corrected stimulus timestamp and appends per-frame AU12 observations `{timestamp_s, intensity}` to `_au12_series` for downstream reward computation.

In parallel, audio bytes accumulate until a full 30-second window is available. The Orchestrator Container then drains the event deque, computes deterministic `segment_window_start_utc`, `segment_window_end_utc`, and `segment_id` values, captures a pre-update BanditDecisionSnapshot at arm-selection time, assembles an InferenceHandoffPayload, attaches `_active_arm`, `_experiment_id`, `_expected_greeting`, `_stimulus_time`, `_au12_series`, `_x_max`, and `_bandit_decision_snapshot`, base64-encodes binary fields for Celery JSON transport, and dispatches the segment to Module D.

4.C.3 AUDIO RESAMPLING

AudioResampler launches a persistent FFmpeg subprocess inside the Orchestrator Container to convert the raw 48 kHz mono IPC stream at `/tmp/ipc/audio_stream.raw` into 16 kHz mono s16le PCM emitted on `pipe:1`.

The read loop runs at video cadence so audio consumption stays synchronized with 30 FPS segment assembly.

If the FFmpeg subprocess is not running, `read_chunk()` waits 1 second and starts a fresh resampler. EOF or pipe read errors stop the old process so the next iteration can recover automatically.

AUDIO CHUNKING IS 1/30 SECOND, NOT 1 SECOND

Consume audio in 1/30-second reads. With `bytes_per_second` = 32000, set `chunk_size` = `int(bytes_per_second / 30)` so each iteration reads 1,066 bytes (approximately 1,067), not a 1-second 32000-byte chunk.

4.C.4 PHYSIOLOGICAL STATE BUFFER

Module C keeps an in-memory Physiological State Buffer so 30-second inference segments can use wearable data that arrives asynchronously and at irregular cadence.

The Orchestrator drains chunk events from Redis, updates a rolling per-subject buffer, derives a current scalar snapshot over the trailing derivation window, and may attach the resulting physiological context to the outgoing segment payload.

- Store a time-ordered deque per `subject_role` covering at least `PHYSIO_BUFFER_RETENTION_S` seconds (default 900). Drain at most 100 events per cycle from `physio:events`.

- Derive each snapshot over `PHYSIO_DERIVE_WINDOW_S` seconds (default 300). Prefer overlapping ibi chunks; otherwise use overlapping session chunks; otherwise `rmssd_ms = null`.
- IBI-derived RMSSD uses `RMSSD = sqrt((1 / (N - 1)) * sum_{i=1}^{N-1}(IBI_{i+1} - IBI_i)^2)`. Heart rate is the rounded mean of overlapping `heart_rate_items_bpm` values or null.
- Compute `validity_ratio = sum(valid_sample_count) / max(sum(expected_sample_count), 1)` and set `is_valid = validity_ratio ⩾ PHYSIO_VALIDITY_MIN` (default 0.80). Invalid windows force `rmssd_ms = null`.
- Set `source_timestamp_utc` from the latest contributing `window_end_utc`, carry `source_kind` and `derivation_method`, and compute `freshness_s` from wall clock time without using ADB drift correction.
- Use `PHYSIO_STALENESS_THRESHOLD_S` (default 600) to classify snapshot age and surface staleness metadata on derived snapshots.

_PHYSIOLOGICAL_CONTEXT IS OPTIONAL

Omit `_physiological_context` entirely when no physiological snapshot exists. Do not emit empty objects, null-filled placeholders, or fabricated defaults.

STALE PHYSIOLOGY DOES NOT BLOCK DISPATCH

When `freshness_s` exceeds `PHYSIO_STALENESS_THRESHOLD_S`, mark `is_stale = True` and continue dispatch. Staleness annotates the payload; it does not block segment assembly or downstream inference.

DRIFT POLLING SPECIFICATION

PARAMETER	VALUE
Polling interval	30 seconds (DRIFT_POLL_INTERVAL)
ADB command	adb shell 'echo $EPOCHREALTIME'
Return format	Decimal epoch string with microsecond precision parsed as float
Maximum tolerated drift	150 milliseconds (MAX_TOLERATED_DRIFT_MS)
Drift formula	drift_offset = host_utc - android_epoch
Timestamp correction	corrected_ts = original_ts + drift_offset
Freeze threshold	3 consecutive ADB failures (DRIFT_FREEZE_AFTER_FAILURES)
Reset timeout	300 seconds / 5 minutes of frozen state (DRIFT_RESET_TIMEOUT)

FFMPEG RESAMPLING COMMAND

FFmpeg resampling command — BASH

```bash
ffmpeg -f s16le -ar 48000 -ac 1 -i /tmp/ipc/audio_stream.raw -ar 16000 -f s16le -ac 1 pipe:1
```

INPUTS	Raw PCM audio from `/tmp/ipc/audio_stream.raw`, H.264 video frames decoded from `/tmp/ipc/video_stream.mkv` by the PyAV-backed video capture thread, ground-truth event dictionaries drained from the in-memory deque, Android epoch timestamps returned by ADB drift polls, and operator stimulus trigger events used to mark greeting injection time.
OUTPUTS	16 kHz resampled mono PCM, drift-corrected segment timestamps, deterministic segment_id, stable segment window bounds in `segment_window_start_utc` and `segment_window_end_utc`, InferenceHandoffPayload segments dispatched as Celery tasks to the ML Worker, per-frame AU12 telemetry in `_au12_series`, stimulus timestamps in `_stimulus_time`, experiment attribution fields `_active_arm`, `_experiment_id`, `_expected_greeting`, `_x_max`, and a pre-update `_bandit_decision_snapshot` captured at arm-selection time.
DEPENDENCIES	FFmpeg persistent subprocess, PyAV-backed video capture, MediaPipe Face Mesh and AU12 normalization via `packages/ml_core`, ADB drift polling, Celery task dispatch, Message Broker (Redis), PostgreSQL access for session registration and Thompson Sampling arm selection, shared Docker volume ipc-share mounted at `/tmp/ipc/`, and data-raw, data-interim, and data-processed volumes.
SIDE EFFECTS	Spawns a persistent FFmpeg subprocess, starts and revives the video capture thread, registers the session in PostgreSQL, selects a Thompson Sampling arm and expected greeting, captures a deterministic pre-update BanditDecisionSnapshot at selection time, computes a stable segment_id from deterministic segment window identity, accumulates AU12 calibration and telemetry state in memory, and dispatches assembled segments to the Celery worker.
FAILURE MODES	1. ADB poll failures freeze the last drift offset after 3 consecutive errors and reset it to zero after 300 seconds of frozen state. 2. Missing or crashed FFmpeg subprocesses wait 1 second before automatic restart, while EOF and read errors stop the old process so the next iteration can recover. 3. Video capture thread death triggers a best-effort stop/recreate/start cycle in the run loop after pipe crashes, AU12 frame processing failures are debug-logged and skipped, event buffer overflow is bounded by deque(maxlen=10000) eviction, and Celery dispatch failures are logged while the loop continues.

4.D Multimodal ML processing

Purpose. Perform speech transcription, observational acoustic analysis, and deterministic semantic evaluation.

Technology stack. faster-whisper v1.2.1, ctranslate2 >=4.5.0, torch 2.4.x, mediapipe 0.10.9, praat-parselmouth v0.4.4, spacy v3.7.x, sentence-transformers v3.2.x, transformers v4.46.x

4.D.1 SPEECH TRANSCRIPTION

Speech transcription is executed by TranscriptionEngine, which wraps faster-whisper with the CTranslate2 backend on the pinned worker runtime.

For each segment, the Celery task stages the 16 kHz s16le PCM bytes through a temporary file and uses FFmpeg to prepare a transcribable audio container before calling `engine.transcribe()`.

GPU execution targets CUDA 12 with cuDNN 8, matching the worker image. cuDNN 8 is required to preserve Pascal SM 6.1 dp4a compatibility for INT8 inference on the target hardware (GTX 1080 Ti).

COMPUTE_TYPE IS FIXED TO INT8

`compute_type` is not a deployment-time knob. TranscriptionEngine fixes `_COMPUTE_TYPE` to `"int8"` to match the Pascal / cuDNN 8 inference path and prevent accidental misconfiguration.

WRAP RAW PCM IN WAV BEFORE TRANSCRIPTION

Before calling `engine.transcribe()`, wrap the temporary 16 kHz s16le PCM bytes in a WAV container with FFmpeg. Headerless PCM is not the intended direct ASR input.

4.D.2 LEGACY COMPUTER VISION COMPATIBILITY NOTE

Canonical AU12 extraction and MediaPipe Face Mesh inference are owned by Module C in v3.4 so per-frame telemetry can be attached to InferenceHandoffPayload before ML Worker dispatch.

Module D must not re-run computer-vision inference for the live reward path. It may consume already-derived AU12 telemetry carried on the handoff payload for validation, replay, or compatibility diagnostics, but any such use is observational only under the §7B reward-path invariance rule.

4.D.3 ACOUSTIC ANALYSIS

The Praat engine accessed through parselmouth extracts observational acoustic analytics from speech audio. Module D computes canonical acoustic summaries and deterministic null-gated validity metadata according to §7D.

For each segment with non-null `_stimulus_time`, Module D must derive stimulus-locked baseline and measurement windows using the same temporal boundaries defined in §7B, perform voiced-only masking and perturbation-span selection according to §7D, and emit the canonical §7D observational acoustic analytics fields to Module E.

If `_stimulus_time` is null, Module D must emit the false/null acoustic outputs defined in §7D.

This analytics path is observational only under the §7B reward-path invariance rule.

4.D.4 TEXT PRE-PROCESSING

spaCy performs tokenization, normalization, and linguistic preprocessing prior to deterministic semantic scoring.

In v3.4 the primary semantic scorer is a local Cross-Encoder executed in the ML Worker. Azure OpenAI is retained only as an optional deterministic gray-band fallback under §8 when the calibrated primary score falls inside a configured ambiguity interval.

INPUTS	16 kHz audio stream and InferenceHandoffPayload segments containing Module C-derived AU12 telemetry.
OUTPUTS	Speech transcripts, semantic evaluation JSON results containing `reasoning`, `is_match`, and `confidence_score` where `reasoning` is a bounded semantic reason code and `confidence_score` is the active method's semantic probability estimate, and the canonical §7D observational acoustic analytics fields. Reward computation continues downstream on the §7B path and this analytics path is observational only under the §7B reward-path invariance rule.
DEPENDENCIES	faster-whisper 1.2.1 with CTranslate2, praat-parselmouth 0.4.4, spaCy 3.7.*, sentence-transformers 3.2.*, transformers 4.46.*, PyTorch 2.4.*, optional Azure OpenAI fallback via the OpenAI Python SDK for gray-band semantic evaluation, FFmpeg for WAV container wrapping, and CUDA 12.2.2 runtime with cuDNN 8.
SIDE EFFECTS	Loads ML models into GPU memory, executes the primary local deterministic semantic scorer, may perform external HTTPS calls to Azure OpenAI only when gray-band fallback is enabled, and emits canonical observational acoustic analytics plus semantic outputs to Module E. This analytics path is observational only under the §7B reward-path invariance rule.
FAILURE MODES	1. GPU OOM triggers worker restart. 2. Whisper load failure aborts startup. 3. Local semantic scorer failure triggers deterministic fallback routing when `llm_gray_band` is enabled; if fallback is unavailable or times out, Module D records a bounded semantic error reason code and deterministic false/null-compatible outputs according to §8. 4. Insufficient voiced coverage, invalid perturbation spans, undefined Praat outputs, and local acoustic extraction exceptions are data-quality conditions; Module D catches them locally and emits deterministic validity flags and null dependent acoustic fields.

4.E Experimentation & analytics

Purpose. Module E aggregates inference outputs into persistent analytics, computes rewards, updates Thompson Sampling state, persists attribution analytics, and serves operator-facing analytics through FastAPI-backed PySide6 surfaces.

Technology stack. SQLite WAL single-writer transactions, pandas v2.2.x, SciPy, RewardResult computation, Thompson Sampling posterior updates, posterior delta emission

4.E.1 EXECUTION DETAILS

Module E aggregates high-frequency inference outputs, reward analytics, physiological/co-modulation metrics, and observational acoustic analytics into structured time-series records.

Reward computation continues to follow §7B and returns a RewardResult dataclass from the established reward path.

Module E persists observational acoustic analytics emitted by Module D to the Persistent Store metrics path and may expose them through operator-facing analytics read APIs. This analytics path is observational only under the §7B reward-path invariance rule.

Operator visualization now runs on the operator host through two PySide6 surfaces, while all data access continues to flow through the FastAPI server.

- `scripts/debug_studio.py`: single-file developer diagnostic GUI for live video, landmarks, AU12 normalization, and analytics polling.
- `services/operator_console/`: modular production console for session monitoring, experiment readback, physiology/co-modulation views, and operator actions.

REWARD ENTRY POINT REMAINS COMPUTE_REWARD()

Physiological, observational acoustic, semantic-probability, and attribution extensions do not change §7B reward computation. `compute_reward()` remains the entry point and still consumes `_au12_series`, `_stimulus_time`, and `is_match`.

4.E.2 PHYSIOLOGICAL CO-MODULATION ANALYTICS

Module E computes rolling physiological co-modulation between the operator and the streamer using valid RMSSD snapshots derived in Module C.

A positive Pearson correlation indicates autonomic alignment and a negative value indicates divergence; the metric is stored as analytics output alongside other session measurements.

- Persist one physiological snapshot per subject per segment to `physiology_log`, including RMSSD, heart rate, validity, freshness, staleness, provider, `source_kind`, and `derivation_method`.
- Resample eligible RMSSD snapshots to 1-minute bins and compute Pearson correlation over a 10-minute rolling window.
- Coverage and variance checks determine whether a Pearson value can be emitted for a window.
- Persist `session_id`, window metadata, Pearson `r`, pair count, coverage ratio, and per-subject RMSSD means to `comodulation_log`.

CO-MODULATION IS OBSERVATIONAL ONLY

This analytics path is observational only under the §7B reward-path invariance rule.

NULL CO-MODULATION IS AN EXPECTED RESULT

A null co-modulation value is valid when there are fewer than 4 paired observations, zero variance in either aligned series, or insufficient eligible overlap. Record the null and continue; do not treat it as a pipeline error.

4.E.3 ATTRIBUTION ANALYTICS

Module E persists a derived attribution ledger that standardizes event identity, delayed outcome capture, deterministic event→outcome eligibility linking, and method-specific attribution scores.

Attribution analytics consume existing inference outputs, semantic evaluation outputs, AU12-derived reward diagnostics, physiological/co-modulation summaries, and delayed outcome events. This analytics path is observational only under the §7B reward-path invariance rule.

- Persist AttributionEvent, OutcomeEvent, EventOutcomeLink, and AttributionScore records.
- Persist only derived and versioned attribution artifacts; raw biometric media, raw physiological chunks, and free-form semantic rationales must not persist.
- Treat missing outcomes, closed attribution horizons, and null linkage as analytical states rather than worker failures.

> **ATTRIBUTION IS ANALYTICAL ONLY IN V3.4**
> Attribution analytics are observational only under the §7B reward-path invariance rule.

MODULE E — FORMAL CONTRACT **§ 4.E.contract**

INPUTS	Inference payloads from Module D delivered through the Celery task queue, plus delayed outcome records ingested through the analytics path for attribution finalization.
OUTPUTS	Time-series records stored in the Persistent Store, including canonical observational acoustic analytics, physiological/co-modulation analytics, AttributionEvent / OutcomeEvent / EventOutcomeLink / AttributionScore records, Thompson Sampling experiment assignments and posterior updates (α `+= r_t`, β `+= 1 − r_t`), RewardResult dataclass produced by `compute_reward()` (§7B), and operator-facing analytics and session-state read APIs consumed by PySide6 host surfaces.
DEPENDENCIES	psycopg2-binary, pandas, SciPy, Message Broker (Redis), Persistent Store (PostgreSQL).
SIDE EFFECTS	Writes metrics, canonical observational acoustic analytics, physiological/co-modulation results, attribution ledger records, and reward results to Persistent Store tables, updates Thompson Sampling Beta posterior parameters per encounter, and exposes analytics and experiment state through the FastAPI server for consumption by PySide6 operator surfaces.
FAILURE MODES	1. Persistent Store outage buffers 1000 records before disk overflow. 2. Celery task deserialization errors are logged and discarded.

4.F Context enrichment

Purpose. Perform asynchronous scraping of external web sources to enrich contextual information about the stream.

Technology stack. HTTPS client, OAuth 2.0 Authorization Code with PKCE, Windows Credential Manager via keyring, signed cloud ExperimentBundle synchronization

4.F.1 EXECUTION DETAILS

Context enrichment tasks run asynchronously outside the main inference loops.

Celery workers execute scraping jobs that gather external metadata about creators or trending topics.

patchright is used instead of standard Playwright to bypass bot-detection mechanisms by patching Chromium automation fingerprints.

MODULE F — FORMAL CONTRACT **§ 4.F.contract**

INPUTS	Celery tasks containing target URL, `scrape_type`, and `timeout_seconds` parameters.
OUTPUTS	JSON objects containing scraped metadata including `source_url`, `scraped_at_utc` timestamp, and extracted data dictionary.
DEPENDENCIES	Celery, Redis Message Broker, patchright browser automation toolkit, Chromium runtime.
SIDE EFFECTS	Launches ephemeral headless browser instances and performs outbound HTTP requests.
FAILURE MODES	1. CAPTCHA or HTTP blocking returns empty result. 2. Browser crash triggers retry. 3. Timeout terminates browser. 4. Broker outage triggers Celery reconnection.

SECTION 5

Data governance and security

LSIE-MLF v4.0 processes sensitive speech, facial-affect, physiological, and behavioral data on a Windows desktop host. The system enforces local data minimization, volatile-media handling, SQLite derived-state limits, cloud schema-on-write rejection, encryption, and retention controls.

5.1 Ephemeral vault specification

PARAMETER	SPECIFICATION
Raw media residency	Process memory, FFmpeg pipes, or SharedMemory only
SharedMemory lifecycle	Owner zeroizes, closes, and unlinks after acknowledgement
SQLite mode	WAL with single writer
Cloud perimeter	Derived telemetry only
Secrets	Windows Credential Manager

5.2 Data classification

Transient media

PCM windows, H.264 MKV stream, decoded frames

STORAGE RULE Allowed only in process memory, FFmpeg pipes, or SharedMemory; never SQLite/cloud/logs.

Local derived state

metrics, posterior state, attribution, physiology summaries, pending uploads

STORAGE RULE SQLite WAL, written only by analytics_state_worker.

Cloud derived telemetry

segment telemetry, posterior deltas, experiment outcomes

STORAGE RULE Cloud PostgreSQL through HTTPS ingest API with schema-on-write validation.

Secrets

OAuth refresh token, cloud client credentials

STORAGE RULE Windows Credential Manager via keyring only.

Forbidden persistent content

raw PCM, decoded H.264 frames, raw MKV, raw Oura JSON, complete PhysiologicalChunkEvent, free-form semantic rationales

STORAGE RULE must be rejected locally and by cloud API models.

The system must not store raw facial images, decoded video frames, raw H.264/MKV media, raw PCM, voiceprints, raw Oura webhook JSON, hydrated provider response bodies, complete transient PhysiologicalChunkEvent payload bodies, or free-form semantic rationales in SQLite, cloud PostgreSQL, logs, crash dumps, pending upload queues, or operator exports. Persistent storage is limited to derived, normalized outputs such as AU12 metrics, scalar RMSSD, heart rate, bounded semantic reason codes, semantic probability estimates, co-modulation metrics, acoustic metrics, posterior deltas, and attribution analytics.

SECTION 6

Interface contracts and payload schemas

Distributed multimodal pipelines require strict interface contracts. v4.0 keeps the canonical InferenceHandoffPayload semantics while changing desktop audio transport from base64 JSON/Celery to Queue-plus-SharedMemory IPC.

LSIE-MLF uses JSON Schema Draft 07 and Pydantic validation to keep module boundaries deterministic at runtime.

The fields _active_arm, _experiment_id, _expected_greeting, _stimulus_time, _au12_series, _x_max, _bandit_decision_snapshot, and _physiological_context survive byte-identically from v3.4.

On the desktop IPC path, _audio_data is validated as part of the producer-side handoff model but is not serialized into the Queue control message. Audio bytes are staged in SharedMemory and referenced by shared_memory_name, byte_length, and checksum.

BanditDecisionSnapshot.random_seed and decision_context_hash are required in v4.0 so offline arm selection is deterministic and replayable across hosts.

6.1 InferenceHandoffPayload

InferenceHandoffPayload JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "InferenceHandoffPayload",
 "description": "Canonical v4.0 InferenceHandoffPayload. Analytical fields are unchanged from
 ↪  v3.4; desktop audio bytes are transported through SharedMemory outside Queue control
 ↪  JSON.",
 "type": "object",
 "required": [
  "session_id",
  "segment_id",
  "segment_window_start_utc",
  "segment_window_end_utc",
  "timestamp_utc",
  "media_source",
  "segments",
  "_active_arm",
  "_experiment_id",
  "_expected_greeting",
  "_stimulus_time",
  "_au12_series",
  "_x_max",
  "_bandit_decision_snapshot"
 ],
 "properties": {
  "session_id": {
   "type": "string",
   "pattern": "^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$",
   "description": "UUID v4 representing the continuous live stream session."
```

```
    },
    "segment_id": {
     "type": "string",
     "pattern": "^[0-9a-f]{64}$",
     "description": "Deterministic SHA-256 segment identifier derived from stable segment identity
     ↪  fields. MUST remain stable across replay and backfill."
    },
    "segment_window_start_utc": {
     "type": "string",
     "format": "date-time",
     "description": "RFC 3339 UTC timestamp of the 30-second segment start boundary."
    },
    "segment_window_end_utc": {
     "type": "string",
     "format": "date-time",
     "description": "RFC 3339 UTC timestamp of the 30-second segment end boundary."
    },
    "timestamp_utc": {
     "type": "string",
     "format": "date-time",
     "description": "RFC 3339 UTC timestamp of inference event completion."
    },
    "media_source": {
     "type": "object",
     "required": [
      "stream_url",
      "codec",
      "resolution"
     ],
     "properties": {
      "stream_url": {
       "type": "string",
       "format": "uri"
      },
      "codec": {
       "type": "string",
       "enum": [
        "h264",
        "h265",
        "raw"
       ]
      },
      "resolution": {
       "type": "array",
       "items": {
        "type": "integer",
        "minimum": 1
       },
       "minItems": 2,
       "maxItems": 2,
       "description": "[width, height] in pixels."
      }
     },
     "additionalProperties": false
    },
    "segments": {
     "type": "array",
     "description": "Array of analyzed video segments."
    },
    "_active_arm": {
     "type": "string",
```

```
      "description": "Thompson Sampling arm identifier selected for this segment."
    },
    "_experiment_id": {
      "type": "integer",
      "description": "Persistent Store experiment row ID for this session."
    },
    "_expected_greeting": {
      "type": "string",
      "description": "Greeting rule text assigned to the active arm for semantic evaluation."
    },
    "_stimulus_time": {
      "type": [
        "number",
        "null"
      ],
      "description": "Drift-corrected UTC epoch seconds of stimulus injection, or null if no
      ↪  stimulus occurred in this segment."
    },
    "_au12_series": {
      "type": "array",
      "items": {
        "type": "object",
        "required": [
          "timestamp_s",
          "intensity"
        ],
        "properties": {
          "timestamp_s": {
            "type": "number"
          },
          "intensity": {
            "type": "number",
            "minimum": 0.0,
            "maximum": 1.0
          }
        },
        "additionalProperties": false
      },
      "description": "Per-frame AU12 bounded intensity observations from the Orchestrator
      ↪  Container."
    },
    "_x_max": {
      "type": [
        "number",
        "null"
      ],
      "description": "Per-subject maximum raw D_mouth/IOD ratio observed during calibration.
      ↪  Carried as calibration telemetry; \u00a77B reward computation consumes bounded AU12
      ↪  intensities and does not divide P90 by this raw ratio."
    },
    "_bandit_decision_snapshot": {
      "$ref": "#/$defs/BanditDecisionSnapshot",
      "description": "Pre-update Thompson Sampling selection snapshot captured in Module C at
      ↪  arm-selection time."
    },
    "_physiological_context": {
      "$ref": "#/$defs/PhysiologicalContext",
      "description": "Optional per-subject physiological context snapshot attached by Module C when
      ↪  eligible physiological data exists."
    },
```

```
    "_desktop_ipc_mode": {
     "type": "boolean",
     "description": "True when the payload is wrapped by a desktop Queue-plus-SharedMemory control
     ↪  message."
    },
    "_audio_transport": {
     "type": "string",
     "enum": [
      "shared_memory",
      "legacy_base64_json"
     ],
     "description": "Desktop path MUST use shared_memory; legacy_base64_json is retained for v3.4
     ↪  replay comparison only."
    }
   },
   "$defs": {
    "ArmPosterior": {
     "type": "object",
     "required": [
      "alpha",
      "beta"
     ],
     "properties": {
      "alpha": {
       "type": "number",
       "exclusiveMinimum": 0.0
      },
      "beta": {
       "type": "number",
       "exclusiveMinimum": 0.0
      }
     },
     "additionalProperties": false
    },
    "BanditDecisionSnapshot": {
     "type": "object",
     "required": [
      "selection_method",
      "selection_time_utc",
      "experiment_id",
      "policy_version",
      "selected_arm_id",
      "candidate_arm_ids",
      "posterior_by_arm",
      "sampled_theta_by_arm",
      "expected_greeting",
      "decision_context_hash",
      "random_seed"
     ],
     "properties": {
      "selection_method": {
       "type": "string",
       "enum": [
        "thompson_sampling"
       ]
      },
      "selection_time_utc": {
       "type": "string",
       "format": "date-time"
      },
      "experiment_id": {
```

```
      "type": "integer"
     },
     "policy_version": {
      "type": "string"
     },
     "selected_arm_id": {
      "type": "string"
     },
     "candidate_arm_ids": {
      "type": "array",
      "items": {
       "type": "string"
      },
      "minItems": 1,
      "uniqueItems": true
     },
     "posterior_by_arm": {
      "type": "object",
      "minProperties": 1,
      "additionalProperties": {
       "$ref": "#/$defs/ArmPosterior"
      }
     },
     "sampled_theta_by_arm": {
      "type": "object",
      "additionalProperties": {
       "type": "number",
       "minimum": 0.0,
       "maximum": 1.0
      }
     },
     "expected_greeting": {
      "type": "string"
     },
     "decision_context_hash": {
      "type": "string",
      "pattern": "^[0-9a-f]{64}$",
      "description": "SHA-256 hash over the stable decision context used for deterministic
      ↪  offline arm selection and replay validation."
     },
     "random_seed": {
      "type": "integer",
      "minimum": 0,
      "maximum": 18446744073709551615,
      "description": "Unsigned 64-bit deterministic seed derived from SHA256(session_id ||
      ↪  segment_window_start_utc || stimulus_time || policy_version)."
     }
    },
    "additionalProperties": false
   },
   "PhysiologicalSnapshot": {
    "type": "object",
    "required": [
     "source_timestamp_utc",
     "freshness_s",
     "is_stale",
     "provider",
     "source_kind",
     "derivation_method",
     "window_s",
     "validity_ratio",
```

```
      "is_valid"
     ],
     "properties": {
      "rmssd_ms": {
       "type": [
        "number",
        "null"
       ],
       "minimum": 0.0
      },
      "heart_rate_bpm": {
       "type": [
        "integer",
        "null"
       ],
       "minimum": 20,
       "maximum": 300
      },
      "source_timestamp_utc": {
       "type": "string",
       "format": "date-time"
      },
      "freshness_s": {
       "type": "number",
       "minimum": 0.0
      },
      "is_stale": {
       "type": "boolean"
      },
      "provider": {
       "type": "string"
      },
      "source_kind": {
       "type": "string",
       "enum": [
        "ibi",
        "session"
       ]
      },
      "derivation_method": {
       "type": "string",
       "enum": [
        "provider",
        "server"
       ]
      },
      "window_s": {
       "type": "integer",
       "exclusiveMinimum": 0
      },
      "validity_ratio": {
       "type": "number",
       "minimum": 0.0,
       "maximum": 1.0
      },
      "is_valid": {
       "type": "boolean"
      }
     },
     "additionalProperties": false
    },
```

```
    "PhysiologicalContext": {
     "type": "object",
     "properties": {
      "streamer": {
       "oneOf": [
        {
         "type": "null"
        },
        {
         "$ref": "#/$defs/PhysiologicalSnapshot"
        }
       ]
      },
      "operator": {
       "oneOf": [
        {
         "type": "null"
        },
        {
         "$ref": "#/$defs/PhysiologicalSnapshot"
        }
       ]
      }
     },
     "additionalProperties": false,
     "anyOf": [
      {
       "required": [
        "streamer"
       ],
       "properties": {
        "streamer": {
         "$ref": "#/$defs/PhysiologicalSnapshot"
        }
       }
      },
      {
       "required": [
        "operator"
       ],
       "properties": {
        "operator": {
         "$ref": "#/$defs/PhysiologicalSnapshot"
        }
       }
      }
     ],
     "description": "Optional per-segment physiological context. When present, at least one of
     ↪  streamer or operator MUST contain a real PhysiologicalSnapshot; empty objects and
     ↪  null-only placeholders are invalid."
    }
   },
   "additionalProperties": false
  }
```

6.2 PhysiologicalSampleEvent

PhysiologicalSampleEvent JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "PhysiologicalChunkEvent",
 "type": "object",
 "required": [
  "unique_id",
  "event_type",
  "provider",
  "subject_role",
  "source_kind",
  "window_start_utc",
  "window_end_utc",
  "ingest_timestamp_utc",
  "payload"
 ],
 "properties": {
  "unique_id": {
   "type": "string",
   "format": "uuid"
  },
  "event_type": {
   "type": "string",
   "enum": [
    "physiological_chunk"
   ]
  },
  "provider": {
   "type": "string",
   "enum": [
    "oura"
   ]
  },
  "subject_role": {
   "type": "string",
   "enum": [
    "streamer",
    "operator"
   ]
  },
  "source_kind": {
   "type": "string",
   "enum": [
    "ibi",
    "session"
   ]
  },
  "window_start_utc": {
   "type": "string",
   "format": "date-time"
  },
  "window_end_utc": {
   "type": "string",
   "format": "date-time"
  },
  "ingest_timestamp_utc": {
   "type": "string",
```

```
      "format": "date-time"
     },
     "payload": {
      "type": "object",
      "required": [
       "sample_interval_s",
       "valid_sample_count",
       "expected_sample_count",
       "derivation_method"
      ],
      "properties": {
       "ibi_ms_items": {
        "type": "array",
        "items": {
         "type": "number",
         "exclusiveMinimum": 0.0
        }
       },
       "rmssd_items_ms": {
        "type": "array",
        "items": {
         "type": "number",
         "minimum": 0.0
        }
       },
       "heart_rate_items_bpm": {
        "type": "array",
        "items": {
         "type": "integer",
         "minimum": 20,
         "maximum": 300
        }
       },
       "motion_items": {
        "type": "array",
        "items": {
         "type": "number",
         "minimum": 0.0
        }
       },
       "sample_interval_s": {
        "type": "integer",
        "exclusiveMinimum": 0
       },
       "valid_sample_count": {
        "type": "integer",
        "minimum": 0
       },
       "expected_sample_count": {
        "type": "integer",
        "minimum": 0
       },
       "derivation_method": {
        "type": "string",
        "enum": [
         "provider",
         "server"
        ]
       }
      },
      "additionalProperties": false,
```

```
    "anyOf": [
     {
      "required": [
       "ibi_ms_items"
      ]
     },
     {
      "required": [
       "rmssd_items_ms"
      ]
     },
     {
      "required": [
       "heart_rate_items_bpm"
      ]
     }
    ]
   }
  },
  "additionalProperties": false
}
```

6.3 PhysiologicalContext

PhysiologicalContext JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "PhysiologicalContext",
 "type": "object",
 "properties": {
  "streamer": {
   "oneOf": [
    {
     "type": "null"
    },
    {
     "$ref": "#/$defs/PhysiologicalSnapshot"
    }
   ]
  },
  "operator": {
   "oneOf": [
    {
     "type": "null"
    },
    {
     "$ref": "#/$defs/PhysiologicalSnapshot"
    }
   ]
  }
 },
 "$defs": {
  "PhysiologicalSnapshot": {
   "type": "object",
   "required": [
```

```
      "source_timestamp_utc",
      "freshness_s",
      "is_stale",
      "provider",
      "source_kind",
      "derivation_method",
      "window_s",
      "validity_ratio",
      "is_valid"
    ],
    "properties": {
      "rmssd_ms": {
        "type": [
          "number",
          "null"
        ],
        "minimum": 0.0
      },
      "heart_rate_bpm": {
        "type": [
          "integer",
          "null"
        ],
        "minimum": 20,
        "maximum": 300
      },
      "source_timestamp_utc": {
        "type": "string",
        "format": "date-time"
      },
      "freshness_s": {
        "type": "number",
        "minimum": 0.0
      },
      "is_stale": {
        "type": "boolean"
      },
      "provider": {
        "type": "string"
      },
      "source_kind": {
        "type": "string",
        "enum": [
          "ibi",
          "session"
        ]
      },
      "derivation_method": {
        "type": "string",
        "enum": [
          "provider",
          "server"
        ]
      },
      "window_s": {
        "type": "integer",
        "exclusiveMinimum": 0
      },
      "validity_ratio": {
        "type": "number",
        "minimum": 0.0,
```

```
      "maximum": 1.0
     },
     "is_valid": {
      "type": "boolean"
     }
    },
    "additionalProperties": false
   }
  },
  "additionalProperties": false,
  "anyOf": [
   {
    "required": [
     "streamer"
    ],
    "properties": {
     "streamer": {
      "$ref": "#/$defs/PhysiologicalSnapshot"
     }
    }
   },
   {
    "required": [
     "operator"
    ],
    "properties": {
     "operator": {
      "$ref": "#/$defs/PhysiologicalSnapshot"
     }
    }
   }
  ],
  "description": "Per-segment physiological context. At least one role MUST contain a real
  ↪  PhysiologicalSnapshot; empty objects and null-only placeholders are invalid."
 }
```

6.4 BanditDecisionSnapshot

BanditDecisionSnapshot JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "BanditDecisionSnapshot",
 "type": "object",
 "required": [
  "selection_method",
  "selection_time_utc",
  "experiment_id",
  "policy_version",
  "selected_arm_id",
  "candidate_arm_ids",
  "posterior_by_arm",
  "sampled_theta_by_arm",
  "expected_greeting",
  "decision_context_hash",
  "random_seed"
```

```
    ],
    "properties": {
     "selection_method": {
      "type": "string",
      "enum": [
       "thompson_sampling"
      ]
     },
     "selection_time_utc": {
      "type": "string",
      "format": "date-time"
     },
     "experiment_id": {
      "type": "integer"
     },
     "policy_version": {
      "type": "string"
     },
     "selected_arm_id": {
      "type": "string"
     },
     "candidate_arm_ids": {
      "type": "array",
      "items": {
       "type": "string"
      },
      "minItems": 1,
      "uniqueItems": true
     },
     "posterior_by_arm": {
      "type": "object",
      "minProperties": 1,
      "additionalProperties": {
       "$ref": "#/$defs/ArmPosterior"
      }
     },
     "sampled_theta_by_arm": {
      "type": "object",
      "additionalProperties": {
       "type": "number",
       "minimum": 0.0,
       "maximum": 1.0
      }
     },
     "expected_greeting": {
      "type": "string"
     },
     "decision_context_hash": {
      "type": "string",
      "pattern": "^[0-9a-f]{64}$",
      "description": "SHA-256 hash over the stable decision context used for deterministic offline
      ↪  arm selection and replay validation."
     },
     "random_seed": {
      "type": "integer",
      "minimum": 0,
      "maximum": 18446744073709551615,
      "description": "Unsigned 64-bit deterministic seed derived from SHA256(session_id ||
      ↪  segment_window_start_utc || stimulus_time || policy_version)."
     }
    },
```

```
  "$defs": {
   "ArmPosterior": {
    "type": "object",
    "required": [
     "alpha",
     "beta"
    ],
    "properties": {
     "alpha": {
      "type": "number",
      "exclusiveMinimum": 0.0
     },
     "beta": {
      "type": "number",
      "exclusiveMinimum": 0.0
     }
    },
    "additionalProperties": false
   }
  },
  "additionalProperties": false
 }
```

6.5 AttributionEvent

AttributionEvent JSON

```
 {
  "$schema": "http://json-schema.org/draft-07/schema#",
  "title": "AttributionEvent",
  "type": "object",
  "required": [
   "event_id",
   "session_id",
   "segment_id",
   "event_type",
   "event_time_utc",
   "selected_arm_id",
   "expected_rule_text_hash",
   "semantic_method",
   "semantic_method_version",
   "reward_path_version",
   "bandit_decision_snapshot",
   "finality",
   "schema_version",
   "created_at"
  ],
  "properties": {
   "event_id": {
    "type": "string",
    "format": "uuid",
    "description": "Deterministic UUIDv5 derived from session_id, segment_id, event_type, and
    ↪  reward_path_version. Replay-stable and used for attribution backfill idempotency."
   },
   "session_id": {
    "type": "string",
```

```
      "pattern": "^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$"
    },
    "segment_id": {
      "type": "string",
      "pattern": "^[0-9a-f]{64}$"
    },
    "event_type": {
      "type": "string",
      "enum": [
        "greeting_interaction"
      ]
    },
    "event_time_utc": {
      "type": "string",
      "format": "date-time"
    },
    "stimulus_time_utc": {
      "type": [
        "string",
        "null"
      ],
      "format": "date-time"
    },
    "selected_arm_id": {
      "type": "string"
    },
    "expected_rule_text_hash": {
      "type": "string",
      "pattern": "^[0-9a-f]{64}$"
    },
    "semantic_method": {
      "type": "string",
      "enum": [
        "cross_encoder",
        "llm_gray_band",
        "azure_llm_legacy"
      ]
    },
    "semantic_method_version": {
      "type": "string"
    },
    "semantic_p_match": {
      "type": [
        "number",
        "null"
      ],
      "minimum": 0.0,
      "maximum": 1.0
    },
    "semantic_reason_code": {
      "type": [
        "string",
        "null"
      ]
    },
    "reward_path_version": {
      "type": "string"
    },
    "bandit_decision_snapshot": {
      "$ref": "#/$defs/BanditDecisionSnapshot"
    },
```

```
    "evidence_flags": {
     "type": "array",
     "items": {
      "type": "string"
     },
     "uniqueItems": true
    },
    "finality": {
     "type": "string",
     "enum": [
      "online_provisional",
      "offline_final"
     ]
    },
    "schema_version": {
     "type": "string"
    },
    "created_at": {
     "type": "string",
     "format": "date-time"
    }
   },
   "$defs": {
    "ArmPosterior": {
     "type": "object",
     "required": [
      "alpha",
      "beta"
     ],
     "properties": {
      "alpha": {
       "type": "number",
       "exclusiveMinimum": 0.0
      },
      "beta": {
       "type": "number",
       "exclusiveMinimum": 0.0
      }
     },
     "additionalProperties": false
    },
    "BanditDecisionSnapshot": {
     "type": "object",
     "required": [
      "selection_method",
      "selection_time_utc",
      "experiment_id",
      "policy_version",
      "selected_arm_id",
      "candidate_arm_ids",
      "posterior_by_arm",
      "expected_greeting"
     ],
     "properties": {
      "selection_method": {
       "type": "string",
       "enum": [
        "thompson_sampling"
       ]
      },
      "selection_time_utc": {
```

```
      "type": "string",
      "format": "date-time"
     },
     "experiment_id": {
      "type": "integer"
     },
     "policy_version": {
      "type": "string"
     },
     "selected_arm_id": {
      "type": "string"
     },
     "candidate_arm_ids": {
      "type": "array",
      "items": {
       "type": "string"
      },
      "minItems": 1,
      "uniqueItems": true
     },
     "posterior_by_arm": {
      "type": "object",
      "minProperties": 1,
      "additionalProperties": {
       "$ref": "#/$defs/ArmPosterior"
      }
     },
     "sampled_theta_by_arm": {
      "type": "object",
      "additionalProperties": {
       "type": "number",
       "minimum": 0.0,
       "maximum": 1.0
      }
     },
     "expected_greeting": {
      "type": "string"
     },
     "decision_context_hash": {
      "type": "string",
      "pattern": "^[0-9a-f]{64}$"
     },
     "random_seed": {
      "type": [
       "integer",
       "null"
      ]
     }
    },
    "additionalProperties": false
   }
  },
  "additionalProperties": false
 }
```

6.6 OutcomeEvent

OutcomeEvent JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "OutcomeEvent",
 "type": "object",
 "required": [
  "outcome_id",
  "session_id",
  "outcome_type",
  "outcome_value",
  "outcome_time_utc",
  "source_system",
  "confidence",
  "finality",
  "schema_version",
  "created_at"
 ],
 "properties": {
  "outcome_id": {
   "type": "string",
   "format": "uuid",
   "description": "Deterministic UUIDv5 derived from session_id, outcome_type, outcome_time_utc,
   ↪  source_system, and source_event_ref. Replay-stable and used for attribution backfill
   ↪  idempotency."
  },
  "session_id": {
   "type": "string",
   "pattern": "^[0-9a-f]{8}-[0-9a-f]{4}-4[0-9a-f]{3}-[89ab][0-9a-f]{3}-[0-9a-f]{12}$"
  },
  "outcome_type": {
   "type": "string",
   "enum": [
    "creator_follow"
   ]
  },
  "outcome_value": {
   "type": "number"
  },
  "outcome_time_utc": {
   "type": "string",
   "format": "date-time"
  },
  "source_system": {
   "type": "string"
  },
  "source_event_ref": {
   "type": [
    "string",
    "null"
   ]
  },
  "confidence": {
   "type": "number",
   "minimum": 0.0,
   "maximum": 1.0
  },
  "finality": {
```

```
    "type": "string",
    "enum": [
     "online_provisional",
     "offline_final"
    ]
   },
   "schema_version": {
    "type": "string"
   },
   "created_at": {
    "type": "string",
    "format": "date-time"
   }
  },
  "additionalProperties": false
 }
```

6.7 EventOutcomeLink

EventOutcomeLink JSON

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "EventOutcomeLink",
 "type": "object",
 "required": [
  "link_id",
  "event_id",
  "outcome_id",
  "lag_s",
  "horizon_s",
  "link_rule_version",
  "eligibility_flags",
  "finality",
  "schema_version",
  "created_at"
 ],
 "properties": {
  "link_id": {
   "type": "string",
   "format": "uuid",
   "description": "Deterministic UUIDv5 derived from event_id, outcome_id, and link_rule_version.
   ↪  Replay-stable and used for attribution backfill idempotency."
  },
  "event_id": {
   "type": "string",
   "format": "uuid"
  },
  "outcome_id": {
   "type": "string",
   "format": "uuid"
  },
  "lag_s": {
   "type": "number",
   "minimum": 0.0
  },
```

```
    "horizon_s": {
     "type": "number",
     "exclusiveMinimum": 0.0
    },
    "link_rule_version": {
     "type": "string"
    },
    "eligibility_flags": {
     "type": "array",
     "items": {
      "type": "string"
     },
     "uniqueItems": true
    },
    "finality": {
     "type": "string",
     "enum": [
      "online_provisional",
      "offline_final"
     ]
    },
    "schema_version": {
     "type": "string"
    },
    "created_at": {
     "type": "string",
     "format": "date-time"
    }
   },
   "additionalProperties": false
  }
```

6.8 AttributionScore

AttributionScore JSON

```
  {
   "$schema": "http://json-schema.org/draft-07/schema#",
   "title": "AttributionScore",
   "type": "object",
   "required": [
    "score_id",
    "event_id",
    "attribution_method",
    "method_version",
    "score_raw",
    "finality",
    "schema_version",
    "created_at"
   ],
   "properties": {
    "score_id": {
     "type": "string",
     "format": "uuid",
```

```
    "description": "Deterministic UUIDv5 derived from event_id, outcome_id, attribution_method,
    ↪  method_version, and finality. Replay-stable and used for attribution backfill
    ↪  idempotency."
   },
   "event_id": {
    "type": "string",
    "format": "uuid"
   },
   "outcome_id": {
    "type": [
     "string",
     "null"
    ],
    "format": "uuid"
   },
   "attribution_method": {
    "type": "string"
   },
   "method_version": {
    "type": "string"
   },
   "score_raw": {
    "type": [
     "number",
     "null"
    ]
   },
   "score_normalized": {
    "type": [
     "number",
     "null"
    ]
   },
   "confidence": {
    "type": [
     "number",
     "null"
    ],
    "minimum": 0.0,
    "maximum": 1.0
   },
   "evidence_flags": {
    "type": "array",
    "items": {
     "type": "string"
    },
    "uniqueItems": true
   },
   "finality": {
    "type": "string",
    "enum": [
     "online_provisional",
     "offline_final"
    ]
   },
   "schema_version": {
    "type": "string"
   },
   "created_at": {
    "type": "string",
    "format": "date-time"
```

```
    }
  },
  "additionalProperties": false
}
```

SECTION 7

Mathematical specifications

7A AU12 facial action unit geometry

AU12 measures lip-corner pull driven by the zygomaticus major. In LSIE-MLF it is the primary facial-affect signal used for reward estimation.

Module C computes AU12 intensity from the 478 three-dimensional MediaPipe Face Mesh landmarks. Geometric normalization uses interocular distance (IOD) as the scale reference so the metric remains stable across head distance and pose.

7A.1 VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
L_i	`landmarks[i]`	3D vector (x_i, y_i, z_i) of MediaPipe landmark i.
L_{61}	`landmarks[61]`	Left lip corner landmark.
L_{291}	`landmarks[291]`	Right lip corner landmark.
L_{33}	`landmarks[33]`	Left eye outer corner.
L_{133}	`landmarks[133]`	Left eye inner corner.
L_{362}	`landmarks[362]`	Right eye inner corner.
L_{263}	`landmarks[263]`	Right eye outer corner.
E_L	`e_l`	Geometric center of left eye midpoint(L_33,L_133).
E_R	`e_r`	Geometric center of right eye midpoint(L_362,L_263).
IOD	`iod`	3D Euclidean distance between E_L and E_R.
D_{mouth}	`d_mouth`	3D Euclidean distance between lip corners.
B_{neutral}	`self.b_neutral`	Neutral baseline average of D_mouth/IOD.
$I_{\text{AU12}}(t)$	`i_au12`	Normalized AU12 intensity at time t.
α	`self.alpha`	Empirical linear projection multiplier. Default 6.0; calibrated so a Duchenne deviation of approximately 0.15 maps to approximately 0.90 before tanh bounding.
$Score_{\text{AU12}}(t)$	`score_au12`	Final AU12 FACS score in range 0–5.
$Score_{\text{AU12,bounded}}(t)$	`compute_bounded_intensity()`	AU12 intensity bounded to [0.0, 1.0] via tanh soft-saturation. Output method for the fractional Beta-Bernoulli Thompson Sampling reward pipeline.

STEP 1: TOPOLOGICAL LANDMARK EXTRACTION

The anatomical points required for AU12 computation are extracted from the MediaPipe landmark array.

- Mouth corners: L_61 (left) and L_291 (right)
- Left eye bounds: L_33 outer and L_133 inner
- Right eye bounds: L_362 inner and L_263 outer

STEP 2: DERIVATION OF INTEROCULAR DISTANCE (IOD)

Eye centers are computed as midpoints between inner and outer eye landmarks and used to calculate the normalization scalar.

$$E_L = ((x_{33}+x_{133})/2, (y_{33}+y_{133})/2, (z_{33}+z_{133})/2); \quad E_R = ((x_{362}+x_{263})/2, (y_{362}+y_{263})/2, (z_{362}+z_{263})/2)$$

$$IOD = \sqrt{(E_R.x - E_L.x)^2 + (E_R.y - E_L.y)^2 + (E_R.z - E_L.z)^2}$$

STEP 3: AU12 DISTANCE AND BASELINE CALIBRATION

The raw AU12 distance equals the Euclidean distance between lip corners.

A neutral baseline B_neutral is computed from the rolling average of D_mouth/IOD during the calibration phase.

The bounded output maps [0, +infinity) to [0, 1) via tanh soft-saturation, providing a continuous gradient at the boundaries suitable for fractional Beta-Bernoulli posterior updates. With alpha_scale=6.0, a Duchenne smile (deviation approximately 0.15) maps to tanh(0.9) approximately 0.72, and laughter (deviation approximately 0.20) maps to tanh(1.2) approximately 0.83.

$$D_{\text{mouth}} = \sqrt{(x_{291} - x_{61})^2 + (y_{291} - y_{61})^2 + (z_{291} - z_{61})^2}$$

$$I_{\text{AU12}}(t) = (D_{\text{mouth}}(t)/IOD(t)) - B_{\text{neutral}}$$

$$Score_{\text{AU12}}(t) = \min\left(5, \max(0, \alpha \cdot I_{\text{AU12}}(t))\right)$$

$$Score_{\text{AU12,bounded}}(t) = \tanh\big(\alpha \cdot \max(0,\ I_{\text{AU12}})\big)$$

7A.2 AU12NORMALIZER — REFERENCE IMPLEMENTATION

Reference implementation **PYTHON**

```python
from __future__ import annotations
import math
from typing import Any
import numpy as np
import numpy.typing as npt

EPSILON: float = 1e-6
```

```python
DEFAULT_ALPHA_SCALE: float = 6.0  # §7A: 0.90 / 0.15 = 6.0


class AU12Normalizer:
    def __init__(self, alpha: float = DEFAULT_ALPHA_SCALE) -> None:
        self.alpha: float = alpha
        self.b_neutral: float | None = None
        self.calibration_buffer: list[float] = []

    def _extract_geometry(
        self, landmarks: npt.NDArray[np.floating[Any]]
    ) -> tuple[float, float] | None:
        left_eye_outer = landmarks[33]
        left_eye_inner = landmarks[133]
        right_eye_inner = landmarks[362]
        right_eye_outer = landmarks[263]
        left_lip = landmarks[61]
        right_lip = landmarks[291]

        e_l = (left_eye_outer + left_eye_inner) / 2.0
        e_r = (right_eye_inner + right_eye_outer) / 2.0
        iod: float = float(np.linalg.norm(e_r - e_l))
        if iod < EPSILON:
            return None
        d_mouth: float = float(np.linalg.norm(right_lip - left_lip))
        return (iod, d_mouth)

    def _update_calibration(self, ratio: float) -> None:
        self.calibration_buffer.append(ratio)
        self.b_neutral = float(np.mean(self.calibration_buffer))

    def compute_raw_ratio(
        self, landmarks: npt.NDArray[np.floating[Any]]
    ) -> float | None:
        geom = self._extract_geometry(landmarks)
        if geom is None:
            return None
        iod, d_mouth = geom
        return d_mouth / iod

    def compute_intensity(
        self, landmarks: npt.NDArray[np.floating[Any]], is_calibrating: bool = False
    ) -> float:
        geom = self._extract_geometry(landmarks)
        if geom is None:
            return 0.0
        iod, d_mouth = geom
        ratio = d_mouth / iod
        if is_calibrating:
            self._update_calibration(ratio)
            return 0.0
        if self.b_neutral is None:
            raise ValueError("Baseline not calibrated")
        score = self.alpha * (ratio - self.b_neutral)
        return float(min(max(score, 0.0), 5.0))

    def compute_bounded_intensity(
        self, landmarks: npt.NDArray[np.floating[Any]], is_calibrating: bool = False
    ) -> float:
        geom = self._extract_geometry(landmarks)
        if geom is None:
```

```
            return 0.0
        iod, d_mouth = geom
        ratio = d_mouth / iod
        if is_calibrating:
            self._update_calibration(ratio)
            return 0.0
        if self.b_neutral is None:
            raise ValueError("Baseline not calibrated")
        deviation = max(0.0, ratio - self.b_neutral)
        return float(math.tanh(self.alpha * deviation))
```

7B Thompson sampling reward pipeline

LSIE-MLF uses Thompson Sampling to select greeting strategies that maximize creator engagement. The reward is derived from AU12 smile intensity (§7A), which serves as a dense proxy for the sparse creator-follow event.

The reward pipeline applies a binary semantic gate so only encounters where the greeting was actually delivered contribute to posterior updates. Non-attributable smiles do not update the posterior.

§7B is the sole normative reward path for v4.0. Physiological co-modulation (§7C), observational acoustic analytics (§7D), semantic probability scores beyond the binary gate (§8), and attribution analytics (§7E) are observational only and must not modify compute_reward(), RewardResult, Thompson Sampling posterior updates, or arm-selection logic.

Transport invariance. The reward calculation is invariant to runtime transport. Whether the segment arrives through the v3.4 Celery/base64 path or the v4.0 Queue-plus-SharedMemory desktop path, analytics_state_worker must compute the same RewardResult for the same validated InferenceHandoffPayload. Gate 0 replay fixtures are the conformance oracle.

7B.1 VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
t_{stim}	`stimulus_time_s`	Drift-corrected UTC epoch timestamp of greeting stimulus injection.
W_{measure}	`extract_stimulus_window()`	Stimulus-locked measurement window [*t_stim* + 0.5s, *t_stim* + 5.0s].
W_{baseline}	`extract_stimulus_-window(baseline)`	Pre-stimulus baseline window [*t_stim* - 5.0s, *t_stim* - 2.0s].
P_{90}	`p90_intensity`	90th percentile of bounded AU12 intensities within *W_measure*.
G_t	`semantic_gate`	Binary semantic validity gate: 1 if `is_match` is True, 0 otherwise.
r_t	`gated_reward`	Gated reward signal: *r_t* = *P_90* × *G_t*. Range [0.0, 1.0].
α	`posterior_alpha`	Beta distribution alpha parameter (success count). Prior: 1.0.
β	`posterior_beta`	Beta distribution beta parameter (failure count). Prior: 1.0.
$B_{\text{neutral,pre}}$	`au12_baseline_pre`	Mean bounded AU12 intensity in *W_baseline*, or null when no pre-stimulus frames exist.
$seed_t$	`random_seed`	Unsigned 64-bit deterministic seed derived as int.from_bytes(SHA256(session_id || segment_window_start_utc || stimulus_time || policy_version).digest()[:8], 'big').

STEP 1: SIGNAL SOURCE AND BOUNDED OUTPUT

The reward pipeline consumes per-frame bounded AU12 intensities from `compute_bounded_intensity()` (§7A). Each observation is a `{timestamp_s, intensity}` pair with intensity in [0.0, 1.0].

STEP 2: STIMULUS WINDOW EXTRACTION

The measurement window is defined as [*t_stim* + 0.5s, *t_stim* + 5.0s]. The 0.5-second onset delay accounts for the physiological latency between hearing the greeting and producing a facial response. The 5.0-second end bound captures the full smile trajectory including onset, apex, and initial decay.

A pre-stimulus baseline window [*t_stim* - 5.0s, *t_stim* - 2.0s] is computed for diagnostic purposes. The 2.0-second guard prevents contamination from anticipatory expressions triggered by the operator's preparation to speak.

$$W_{\text{measure}} = \{(t_i, s_i) \in \text{au12_series} \mid t_{\text{stim}} + 0.5 \leq t_i \leq t_{\text{stim}} + 5.0\}$$

$$W_{\text{baseline}} = \{(t_i, s_i) \in \text{au12_series} \mid t_{\text{stim}} - 5.0 \leq t_i \leq t_{\text{stim}} - 2.0\}$$

STEP 3: P90 AGGREGATION AND GATED REWARD

The 90th percentile is used instead of the mean or maximum to reduce sensitivity to single-frame noise spikes while still capturing the peak smile response. If the measurement window contains zero frames, P90 defaults to 0.0.

The semantic gate G_t is binary and comes from the `is_match` output (§8). If the greeting was not delivered, the reward is zero regardless of smile intensity.

$$P_{90} = \text{percentile}_{90}(\{s_i \mid (t_i, s_i) \in W_{\text{measure}}\})$$

$$G_t = \begin{cases} 1 & \text{if is_match} = \text{True} \\ 0 & \text{otherwise} \end{cases}$$

$$r_t = P_{90} \times G_t$$

STEP 4: FRACTIONAL BETA-BERNOULLI POSTERIOR UPDATE

Each arm maintains a Beta posterior initialized with a uniform prior Beta(1, 1). After each encounter, the posterior is updated with the fractional reward $r_t \in [0, 1]$. Unlike standard Bernoulli updates that require binary 0/1 outcomes, the fractional update preserves the continuous gradient of the bounded AU12 signal.

Arm selection uses Thompson Sampling: for each arm, a value is sampled from its current Beta posterior, and the arm with the highest sampled value is selected for the next encounter.

$$\alpha \leftarrow \alpha + r_t, \quad \beta \leftarrow \beta + (1 - r_t)$$

$$a^* = \arg\max_a \theta_a, \quad \theta_a \sim \text{Beta}(\alpha_a, \beta_a)$$

STEP 5: DETERMINISTIC OFFLINE ARM SELECTION SEED

When the cloud control plane is unavailable, offline arm selection remains deterministic and replayable. module_c_orchestrator derives random_seed before selection and stores it in BanditDecisionSnapshot.

The seed derivation uses canonical session_id, segment_window_start_utc, stimulus_time, and policy_version bytes. This seed does not change §7B reward math.

$$seed_t = \text{int64}_{\text{be}}(\text{SHA256}(session_id \| segment_window_start_utc \| stimulus_time \| policy_version)[0:8])$$

7B.2 REWARD PIPELINE AND THOMPSON SAMPLING — REFERENCE IMPLEMENTATION

Reference implementation **PYTHON**

```python
from __future__ import annotations
import random
from dataclasses import dataclass
from typing import Optional
import numpy as np

STIMULUS_WINDOW_START: float = 0.5   # seconds after stimulus
STIMULUS_WINDOW_END: float = 5.0     # seconds after stimulus
BASELINE_WINDOW_START: float = -5.0  # seconds before stimulus
```

```python
BASELINE_WINDOW_END: float = -2.0    # seconds before stimulus
DEFAULT_PRIOR_ALPHA: float = 1.0
DEFAULT_PRIOR_BETA: float = 1.0


@dataclass
class RewardResult:
    gated_reward: float
    p90_intensity: float
    semantic_gate: int
    n_frames_in_window: int
    au12_baseline_pre: Optional[float]


def extract_stimulus_window(
    au12_series: list[dict], stimulus_time_s: float,
    start_offset: float = STIMULUS_WINDOW_START,
    end_offset: float = STIMULUS_WINDOW_END,
) -> list[dict]:
    t_start = stimulus_time_s + start_offset
    t_end = stimulus_time_s + end_offset
    return [o for o in au12_series if t_start ≤ o["timestamp_s"] ≤ t_end]


def compute_reward(
    au12_series: list[dict], stimulus_time_s: float,
    is_match: bool,
) -> RewardResult:
    window = extract_stimulus_window(au12_series, stimulus_time_s)
    n_frames = len(window)

    if n_frames == 0:
        p90 = 0.0
    else:
        p90 = float(np.percentile([o["intensity"] for o in window], 90))


    gate = 1 if is_match else 0
    gated = p90 * gate

    baseline_obs = extract_stimulus_window(
        au12_series, stimulus_time_s,
        start_offset=BASELINE_WINDOW_START, end_offset=BASELINE_WINDOW_END,
    )
    baseline = (
        float(np.mean([o["intensity"] for o in baseline_obs]))
        if baseline_obs else None
    )

    return RewardResult(
        gated_reward=gated, p90_intensity=p90, semantic_gate=gate,
        n_frames_in_window=n_frames, au12_baseline_pre=baseline,
    )


def thompson_sample_arm(arms: dict[str, tuple[float, float]]) -> str:
    samples = {
        arm_id: random.betavariate(alpha, beta)
        for arm_id, (alpha, beta) in arms.items()
    }
    return max(samples, key=samples.get)
```

```python
def update_posterior(
    alpha: float, beta: float, reward: float
) -> tuple[float, float]:
    return alpha + reward, beta + (1.0 - reward)
```

7C Physiological co-modulation analytics

This section defines physiological co-modulation as a rolling Pearson correlation between streamer and operator RMSSD time series.

RMSSD snapshots are derived in Module C from normalized physiological chunks and persisted per segment. This analytics path is observational only under the §7B reward-path invariance rule.

7C.1 VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
$R_s^{(k)}$	`streamer_rmssd_bins[k]`	Streamer RMSSD in a 1-minute resampled analysis bin.
$R_o^{(k)}$	`operator_rmssd_bins[k]`	Operator RMSSD in a 1-minute resampled analysis bin.
C_{mod}	`co_modulation_index`	Pearson correlation between aligned streamer and operator RMSSD sequences.
N	`n_paired_observations`	Number of aligned paired observations in the analysis window.
$coverage$_ratio	`coverage_ratio`	Fraction of the analysis window with valid paired observations.
MIN_COMOD_PAIRS	`MIN_COMOD_PAIRS`	Minimum aligned pairs required for a valid correlation.
$COMOD$_WINDOW_MINUTES	`COMOD_WINDOW_MINUTES`	Rolling analysis window duration in minutes.

STEP 1: DATA SOURCE AND ELIGIBILITY

Use per-segment physiological snapshots from `physiology_log`.

A snapshot is eligible only when `rmssd_ms` is not null, `is_valid = TRUE`, and `is_stale = FALSE`.

STEP 2: WINDOWING, RESAMPLING, AND TIME ALIGNMENT

For each subject, select eligible snapshots in the 10-minute window ending at `window_end_utc`.

Resample each series to 1-minute mean bins and inner-join on timestamp.

$$R_s = \text{resample}_{1\text{min}}(\text{streamer_rmssd})$$

$$R_o = \text{resample}_{1\text{min}}(\text{operator_rmssd})$$

STEP 3: PEARSON CORRELATION AND NULL GATING

Compute Pearson correlation on the aligned RMSSD series. Positive values indicate alignment; negative values indicate divergence.

- If N < 4 (MIN_COMOD_PAIRS), co_modulation_index = null.
- If either aligned series has zero variance, co_modulation_index = null.
- If N >= 4 and both aligned series have non-zero variance, compute co_modulation_index using scipy.stats.pearsonr.

$$C_{\text{mod}} = \frac{\sum(R_s^{(k)} - \bar{R}_s)(R_o^{(k)} - \bar{R}_o)}{\sqrt{\sum(R_s^{(k)} - \bar{R}_s)^2 \sum(R_o^{(k)} - \bar{R}_o)^2}}$$

STEP 4: COVERAGE RATIO AND PERSISTENCE

Coverage ratio measures the fraction of the window with valid paired observations.

Persist the result even when `co_modulation_index` is null.

- co_modulation_index
- n_paired_observations
- coverage_ratio
- streamer_rmssd_mean
- operator_rmssd_mean
- window_start_utc
- window_end_utc

$$\text{coverage_ratio} = \frac{N}{\max(|R_s|, |R_o|, 1)}$$

STEP 5: SCOPE CONSTRAINT

This analytics path is observational only under the §7B reward-path invariance rule.

7C.2 PHYSIOLOGICAL CO-MODULATION — REFERENCE IMPLEMENTATION

Reference implementation **PYTHON**

```python
from __future__ import annotations

from dataclasses import dataclass
from typing import Optional

import pandas as pd
from scipy.stats import pearsonr

MIN_COMOD_PAIRS: int = 4
```

```python
COMOD_WINDOW_MINUTES: int = 10
RESAMPLE_RULE: str = "1min"


@dataclass(frozen=True)
class CoModulationResult:
    co_modulation_index: Optional[float]
    n_paired_observations: int
    coverage_ratio: float
    streamer_rmssd_mean: Optional[float]
    operator_rmssd_mean: Optional[float]


def _resample_rmssd(window_df: pd.DataFrame, subject_role: str) -> pd.Series:
    role_df = (
        window_df.loc[
            window_df["subject_role"] == subject_role,
            ["source_timestamp_utc", "rmssd_ms"],
        ]
        .dropna(subset=["rmssd_ms"])
        .sort_values("source_timestamp_utc")
    )

    if role_df.empty:
        return pd.Series(dtype="float64")

    return (
        role_df.set_index("source_timestamp_utc")["rmssd_ms"]
        .resample(RESAMPLE_RULE)
        .mean()
        .dropna()
    )


def compute_comodulation(
    physiology_log: pd.DataFrame,
    window_end_utc: pd.Timestamp,
) -> CoModulationResult:
    window_end_utc = pd.Timestamp(window_end_utc, tz="UTC")
    window_start_utc = window_end_utc - pd.Timedelta(minutes=COMOD_WINDOW_MINUTES)

    eligible = physiology_log.copy()
    eligible["source_timestamp_utc"] = pd.to_datetime(
        eligible["source_timestamp_utc"], utc=True
    )
    eligible = eligible.loc[
        (eligible["source_timestamp_utc"] > window_start_utc)
        & (eligible["source_timestamp_utc"] ≤ window_end_utc)
        & eligible["rmssd_ms"].notna()
        & (eligible["is_valid"] == True)
        & (eligible["is_stale"] == False)
    ]

    streamer = _resample_rmssd(eligible, "streamer")
    operator = _resample_rmssd(eligible, "operator")

    aligned = pd.concat(
        [streamer.rename("streamer"), operator.rename("operator")],
        axis=1,
        join="inner",
    ).dropna()
```

```
    n_pairs = int(len(aligned))
    coverage_ratio = float(
        n_pairs / max(len(streamer), len(operator), 1)
    )

    streamer_mean = float(streamer.mean()) if not streamer.empty else None
    operator_mean = float(operator.mean()) if not operator.empty else None

    if n_pairs < MIN_COMOD_PAIRS:
        return CoModulationResult(
            co_modulation_index=None,
            n_paired_observations=n_pairs,
            coverage_ratio=coverage_ratio,
            streamer_rmssd_mean=streamer_mean,
            operator_rmssd_mean=operator_mean,
        )

    if aligned["streamer"].nunique() < 2 or aligned["operator"].nunique() < 2:
        return CoModulationResult(
            co_modulation_index=None,
            n_paired_observations=n_pairs,
            coverage_ratio=coverage_ratio,
            streamer_rmssd_mean=streamer_mean,
            operator_rmssd_mean=operator_mean,
        )

    co_modulation_index, _p_value = pearsonr(
        aligned["streamer"], aligned["operator"]
    )

    return CoModulationResult(
        co_modulation_index=float(co_modulation_index),
        n_paired_observations=n_pairs,
        coverage_ratio=coverage_ratio,
        streamer_rmssd_mean=streamer_mean,
        operator_rmssd_mean=operator_mean,
    )
```

7D Observational acoustic analytics

This section defines stimulus-windowed observational acoustic analytics from streamer speech audio.

The metrics are derived in Module D from the existing 16 kHz speech stream and persisted as derived analytical outputs. This analytics path is observational only under the §7B reward-path invariance rule.

7D.1 VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
τ_{voice}	`ACOUSTIC_VOICING_THRESHOLD`	Minimum Praat voicing threshold for a frame to be treated as voiced.
$T_{\text{voice,min}}$	`ACOUSTIC_MIN_VOICED_COVERAGE_S`	Minimum cumulative voiced coverage required for F0 validity inside a window.
$T_{\text{island,min}}$	`ACOUSTIC_MIN_STABLE_ISLAND_S`	Minimum duration of one continuous voiced island required for F0 validity inside a window.
$N_{\text{peaks,min}}$	`ACOUSTIC_MIN_PERIODIC_PEAKS`	Minimum periodic peaks required for perturbation validity inside a selected voiced island.
$V(W)$	`voiced_frames(window)`	Frames in the analysis window whose voicing strength meets or exceeds the voicing threshold.
$C_{\text{voice}}(W)$	`voiced_coverage_s(window)`	Sum of durations of voiced frames in the analysis window.
$L_{\text{voice}}(W)$	`longest_voiced_island_s(window)`	Duration of the longest contiguous voiced island in the analysis window.
$P(W)$	`perturbation_span(window)`	Selected contiguous voiced island used for jitter/shimmer computation; it must satisfy perturbation validity constraints.
Δ_{ST}	`f0_delta_semitones`	Semitone difference between measurement-window and baseline-window mean F0.

STEP 1: STIMULUS WINDOWS AND VOICED-ONLY MASKING

Use the same stimulus-locked windows already defined by §7B.

For each analysis window, mark frames as voiced only when voicing strength is greater than or equal to 0.45, compute cumulative voiced coverage, compute the longest contiguous voiced island, and exclude unvoiced frames from all voiced-only aggregates.

- ACOUSTIC_VOICING_THRESHOLD = 0.45
- ACOUSTIC_MIN_VOICED_COVERAGE_S = 1.0
- ACOUSTIC_MIN_STABLE_ISLAND_S = 0.2
- ACOUSTIC_MIN_PERIODIC_PEAKS = 4

$$W_{\text{measure}} = [t_{\text{stim}} + 0.5\text{s},\ t_{\text{stim}} + 5.0\text{s}]$$

$$W_{\text{baseline}} = [t_{\text{stim}} - 5.0\text{s},\ t_{\text{stim}} - 2.0\text{s}]$$

$$V(W) = \{\text{frame}_i \in W \mid \text{voicing_strength}(\text{frame}_i) \geq \tau_{\text{voice}}\}$$

$$C_{\text{voice}}(W) = \sum_{\text{frame}_i \in V(W)} \text{duration}(\text{frame}_i)$$

$$L_{\text{voice}}(W) = \text{max_contiguous_duration}(V(W))$$

STEP 2: VALIDITY GATING AND PERTURBATION-SPAN SELECTION

For each window W, F0 validity requires minimum cumulative voiced coverage and a minimum stable voiced island.

Perturbation validity requires F0 validity plus at least one contiguous voiced island containing the minimum periodic peaks and computable Praat local jitter/local shimmer values. If multiple eligible perturbation spans exist, use the longest eligible span.

- f0_valid(W) = true iff C_voice(W) >= ACOUSTIC_MIN_VOICED_COVERAGE_S and L_voice(W) >= ACOUSTIC_MIN_STABLE_ISLAND_S
- perturbation_valid(W) = true iff f0_valid(W) is true, at least one contiguous voiced island contains >= ACOUSTIC_MIN_PERIODIC_PEAKS periodic peaks, and Praat local jitter / local shimmer are computable on the selected perturbation span
- Otherwise the corresponding validity flag is false

STEP 3: SUMMARY STATISTICS AND DELTA FORMULAS

Voiced coverage outputs are the cumulative voiced coverage in the measure and baseline windows.

If F0 validity is true, compute arithmetic means only over voiced frames with F0 greater than zero. If perturbation validity is true, compute Praat local jitter and local shimmer on the selected perturbation span.

- If `f0_valid(W) = false`, the corresponding F0 mean is null.
- If `perturbation_valid(W) = false`, the corresponding perturbation means are null.
- If either F0 mean is null, `f0_delta_semitones` is null.
- If `f0_mean_baseline_hz` <= 0, `f0_delta_semitones` is null.
- If either jitter mean is null, `jitter_delta` is null.
- If either shimmer mean is null, `shimmer_delta` is null.
- Session-level z-scoring or population normalization must not be persisted at insertion time. The Persistent Store retains raw means, validity flags, coverage values, and canonical deltas only.

$$f0_mean_measure_hz = \text{mean}(F0(\text{frame}_i) \mid \text{frame}_i \in V(W_{\text{measure}}),\ F0(\text{frame}_i) > 0)$$

$$f0_mean_baseline_hz = \text{mean}(F0(\text{frame}_i) \mid \text{frame}_i \in V(W_{\text{baseline}}),\ F0(\text{frame}_i) > 0)$$

$$\Delta_{\text{ST}} = 12 \cdot \log_2 \left(\frac{f0_mean_measure_hz}{f0_mean_baseline_hz} \right)$$

$$\text{jitter_delta} = \text{jitter_mean_measure} - \text{jitter_mean_baseline}$$

$$\text{shimmer_delta} = \text{shimmer_mean_measure} - \text{shimmer_mean_baseline}$$

STEP 4: NULL STIMULUS BEHAVIOR

If `_stimulus_time` is null, Module D emits deterministic false/null observational acoustic outputs instead of attempting stimulus-window analysis.

- f0_valid_measure = false
- f0_valid_baseline = false
- perturbation_valid_measure = false
- perturbation_valid_baseline = false
- voiced_coverage_measure_s = 0.0
- voiced_coverage_baseline_s = 0.0
- all mean and delta fields = null

STEP 5: SCOPE CONSTRAINT

This analytics path is observational only under the §7B reward-path invariance rule.

7D.2 OBSERVATIONAL ACOUSTIC ANALYTICS — REFERENCE IMPLEMENTATION

Reference implementation PYTHON

```python
from __future__ import annotations

from dataclasses import dataclass
from math import log2
from typing import Optional

ACOUSTIC_VOICING_THRESHOLD: float = 0.45
ACOUSTIC_MIN_VOICED_COVERAGE_S: float = 1.0
ACOUSTIC_MIN_STABLE_ISLAND_S: float = 0.2
ACOUSTIC_MIN_PERIODIC_PEAKS: int = 4

STIMULUS_WINDOW_START: float = 0.5
STIMULUS_WINDOW_END: float = 5.0
BASELINE_WINDOW_START: float = -5.0
BASELINE_WINDOW_END: float = -2.0


@dataclass(frozen=True)
class AcousticAnalyticsResult:
    f0_valid_measure: bool
    f0_valid_baseline: bool
    perturbation_valid_measure: bool
    perturbation_valid_baseline: bool
    voiced_coverage_measure_s: float
    voiced_coverage_baseline_s: float
    f0_mean_measure_hz: Optional[float]
    f0_mean_baseline_hz: Optional[float]
    f0_delta_semitones: Optional[float]
    jitter_mean_measure: Optional[float]
    jitter_mean_baseline: Optional[float]
    jitter_delta: Optional[float]
    shimmer_mean_measure: Optional[float]
```

```
    shimmer_mean_baseline: Optional[float]
    shimmer_delta: Optional[float]


def compute_f0_delta_semitones(
    f0_mean_measure_hz: Optional[float],
    f0_mean_baseline_hz: Optional[float],
) -> Optional[float]:
    if f0_mean_measure_hz is None or f0_mean_baseline_hz is None:
        return None
    if f0_mean_baseline_hz ≤ 0:
        return None
    return 12.0 * log2(f0_mean_measure_hz / f0_mean_baseline_hz)


def null_acoustic_result() -> AcousticAnalyticsResult:
    return AcousticAnalyticsResult(
        f0_valid_measure=False,
        f0_valid_baseline=False,
        perturbation_valid_measure=False,
        perturbation_valid_baseline=False,
        voiced_coverage_measure_s=0.0,
        voiced_coverage_baseline_s=0.0,
        f0_mean_measure_hz=None,
        f0_mean_baseline_hz=None,
        f0_delta_semitones=None,
        jitter_mean_measure=None,
        jitter_mean_baseline=None,
        jitter_delta=None,
        shimmer_mean_measure=None,
        shimmer_mean_baseline=None,
        shimmer_delta=None,
    )
```

7E Event→Outcome attribution policy

This section defines cross-signal attribution as a derived analytical policy layer spanning semantic scoring, baseline-aware AU12 diagnostics, deterministic event→outcome linkage, and lag-aware observational synchrony analytics.

This analytics path is observational only under the §7B reward-path invariance rule.

7E.1 VARIABLE DICTIONARY

SYMBOL	CODE NAME	DEFINITION
$segment_{\text{id}}$	`segment_id`	Deterministic SHA-256 segment identifier derived from stable segment identity fields.
H_{attr}	`attribution_horizon_s`	Maximum eligible event→outcome attribution horizon in seconds.
lag_s	`lag_s`	Time difference between outcome timestamp and event timestamp in seconds.
p_{match}	`confidence_score`	Semantic probability estimate produced by the active deterministic semantic method.
r_t^{soft}	`soft_reward_candidate`	Observational soft semantic reward candidate persisted under §7E only.
B_{pre}	`au12_baseline_pre`	Mean bounded AU12 intensity in the pre-stimulus baseline window.
L_{p90}	`au12_lift_p90`	Baseline-aware lift using the post-stimulus P90 bounded AU12 intensity.
L_{peak}	`au12_lift_peak`	Baseline-aware lift using the post-stimulus peak bounded AU12 intensity.
$\Delta t_{\text{peak,ms}}$	`au12_peak_latency_ms`	Latency from stimulus timestamp to peak AU12 observation in milliseconds.
$\rho(\ell)$	`lagged_correlation`	Lagged Pearson correlation at lag \ell.
S	`sync_peak_corr`	Maximum lagged correlation over the configured lag search range.
$finality$	`finality`	Attribution record lifecycle state: online_provisional or offline_final.
r_t^{proxy}	`proxy_reward`	The observational proxy reward candidate used for attribution scoring, typically the §7B gated_reward or a documented diagnostic reward component.
x_t	`source_series`	First time-aligned observational signal used in lag-aware synchrony analytics.
y_t	`target_series`	Second time-aligned observational signal used in lag-aware synchrony analytics.

STEP 1: STABLE EVENT IDENTITY AND SCOPE

Each dispatched interaction segment must carry a deterministic segment identity so reprocessing and offline linkage remain idempotent.

A stable event key is derived in Module C before dispatch and reused by Module E when constructing AttributionEvent records.

$$\text{segment_id} = \text{SHA256}(\text{session_id} \parallel \text{segment_window_start_utc} \parallel \text{segment_window_end_utc})$$

STEP 2: OUTCOME DEFINITION AND ELIGIBILITY LINKING

Delayed outcomes such as creator_follow are persisted as OutcomeEvent records and linked to prior events by deterministic eligibility rules.

Eligibility linkage is many-to-many and defines candidate event→outcome pairs; it does not assert unique causal proof.

$$\text{lag}_s = t_{\text{outcome}} - t_{\text{event}}$$

$$0 \leq \text{lag}_s \leq H_{\text{attr}}$$

STEP 3: SEMANTIC ATTRIBUTION SCORES

The semantic scorer outputs a probability estimate *p_match* together with a bounded reason code and final boolean gate.

In v3.4 the soft semantic reward is persisted only as an observational candidate and does not modulate §7B reward updates.

$$r_t^{\text{soft}} = p_{\text{match}} \cdot r_t^{\text{proxy}}$$

$$P(\text{match} \mid \text{audio}, \text{rule}) = \sum_{k=1}^{K} P(\hat{u}_k \mid \text{audio})\, P_\phi(\text{match} \mid \hat{u}_k, \text{rule})$$

STEP 4: AU12 BASELINE-AWARE ATTRIBUTION METRICS

To distinguish stimulus-linked smiles from smiles already underway, attribution analytics persist baseline-aware AU12 lift metrics derived from the same windows already defined by §7B.

These metrics are diagnostic and do not replace the §7B gated reward in v3.4.

$$L_{\text{p90}} = \max(0, P_{90,post} - B_{\text{pre}})$$

$$L_{\text{peak}} = \max(0, P_{\text{max,post}} - B_{\text{pre}})$$

$$\Delta t_{\text{peak,ms}} = 1000 \cdot (t_{\text{peak}} - t_{\text{stim}})$$

STEP 5: LAG-AWARE SYNCHRONY ANALYTICS

Before more complex warping methods are introduced, attribution analytics persist a lagged-correlation baseline for observational synchrony.

These synchrony metrics remain observational and do not affect reward computation.

$$\rho(\ell) = \text{corr}(x_t, y_{t+\ell}), \qquad \ell \in [0, L]$$

$$S = \max_{\ell \in [0,L]} \rho(\ell)$$

STEP 6: FINALITY AND SCOPE CONSTRAINT

Attribution records transition from `online_provisional` to `offline_final` after attribution horizons close and finalization jobs complete.

All §7E outputs are analytical only under the §7B reward-path invariance rule.

- Allowed finality states: online_provisional, offline_final.
- Closed horizons do not reopen unless explicit reprocessing is invoked.

7E.2 ATTRIBUTION ANALYTICS — REFERENCE IMPLEMENTATION

Reference implementation PYTHON

```python
from __future__ import annotations

import hashlib
from dataclasses import dataclass
from typing import Optional

import numpy as np
from scipy.stats import pearsonr

STIMULUS_WINDOW_START: float = 0.5
STIMULUS_WINDOW_END: float = 5.0
BASELINE_WINDOW_START: float = -5.0
BASELINE_WINDOW_END: float = -2.0


def compute_segment_id(session_id: str, segment_window_start_utc: str, segment_window_end_utc:
↪  str) -> str:
    raw = f"{session_id}|{segment_window_start_utc}|{segment_window_end_utc}".encode("utf-8")
    return hashlib.sha256(raw).hexdigest()


def compute_soft_reward_candidate(p_match: Optional[float], proxy_reward: float) ->
↪  Optional[float]:
    if p_match is None:
        return None
    p = float(min(max(p_match, 0.0), 1.0))
    return p * float(proxy_reward)


def compute_au12_attribution_metrics(au12_series: list[dict], stimulus_time_s: float) ->
↪  dict[str, Optional[float]]:
    baseline_obs = [
        o["intensity"] for o in au12_series
        if stimulus_time_s + BASELINE_WINDOW_START ≤ o["timestamp_s"] ≤ stimulus_time_s +
        ↪  BASELINE_WINDOW_END
    ]
    measure_obs = [
        o for o in au12_series
        if stimulus_time_s + STIMULUS_WINDOW_START ≤ o["timestamp_s"] ≤ stimulus_time_s +
        ↪  STIMULUS_WINDOW_END
    ]
    if not measure_obs:
        return {
            "au12_baseline_pre": float(np.mean(baseline_obs)) if baseline_obs else None,
            "au12_lift_p90": None,
```

```
            "au12_lift_peak": None,
            "au12_peak_latency_ms": None,
        }

    baseline_pre = float(np.mean(baseline_obs)) if baseline_obs else 0.0
    values = np.array([o["intensity"] for o in measure_obs], dtype=float)
    p90_post = float(np.percentile(values, 90))
    peak_idx = int(np.argmax(values))
    peak_post = float(values[peak_idx])
    peak_latency_ms = float((measure_obs[peak_idx]["timestamp_s"] - stimulus_time_s) * 1000.0)

    return {
        "au12_baseline_pre": baseline_pre,
        "au12_lift_p90": max(0.0, p90_post - baseline_pre),
        "au12_lift_peak": max(0.0, peak_post - baseline_pre),
        "au12_peak_latency_ms": peak_latency_ms,
    }


def compute_peak_lagged_corr(x: np.ndarray, y: np.ndarray, max_lag: int) ->
↪  tuple[Optional[float], Optional[int]]:
    best_r: Optional[float] = None
    best_lag: Optional[int] = None
    for lag in range(0, max_lag + 1):
        if lag == 0:
            x_lag = x
            y_lag = y
        else:
            x_lag = x[:-lag]
            y_lag = y[lag:]
        if len(x_lag) < 4 or len(y_lag) < 4:
            continue
        if np.std(x_lag) == 0.0 or np.std(y_lag) == 0.0:
            continue
        r, _ = pearsonr(x_lag, y_lag)
        if best_r is None or float(r) > best_r:
            best_r = float(r)
            best_lag = lag
    return best_r, best_lag
```

SECTION 8

Deterministic semantic scorer specification

Module D must determine whether a spoken utterance satisfies a predefined semantic rule such as a required greeting phrase. Traditional string-matching algorithms like Levenshtein distance or BLEU fail because human speech contains fillers, paraphrasing, and transcription noise.

To perform robust evaluation while maintaining reproducibility, LSIE-MLF uses a deterministic semantic scorer registry. In v3.4 the primary method is a local Cross-Encoder executed in the ML

Worker. An optional Azure OpenAI gray-band fallback may be invoked only when the calibrated primary score falls inside a configured ambiguity interval.

The `is_match` boolean remains the reward gate $G_t \in \{0, 1\}$. When `is_match` is True, P90 AU12 intensity contributes to the posterior update; otherwise the reward is zero. `confidence_score` is the active method's semantic probability estimate and remains analytical only in v3.4; it does not modulate the §7B reward path.

The `reasoning` field is a bounded semantic reason code, not a free-form rationale. Implementations must not persist free-form semantic rationales.

Semantic shadow mode runs a candidate semantic method in parallel with the active method and records its score for evaluation only. Shadow-mode outputs must not alter live `is_match`, `confidence_score`, reward computation, or Thompson Sampling updates.

8.1 Inference parameter constraints

PARAMETER	VALUE	JUSTIFICATION
`temperature`	`0.0`	Collapses the probability distribution to deterministic token selection.
`top_p`	`1.0`	Disables nucleus sampling ensuring deterministic token selection.
`max_tokens`	`500`	Ensures adequate space for gray-band fallback deliberation plus structured JSON output.
`response_format`	`JSON Schema Structured Outputs`	Forces the model to return machine-parseable JSON rather than conversational text.
`seed`	`42`	Activates Azure OpenAI deterministic fallback mode.
`cross_encoder_model_id`	`local://models/semantic/lsie-greeting-cross-encoder-v1`	Pins the primary local semantic scorer artifact used for greeting-rule matching across deployments.
`cross_encoder_model_version`	`lsie-greeting-cross-encoder-v1.0.0`	Pins the exact primary scorer release used for deterministic replay and audit.
`match_threshold`	`0.72`	Defines the deterministic probability cutoff: scores >= 0.72 are matches outside the gray band.
`gray_band_interval`	`0.58 ⩽ score < 0.72`	Defines the only score interval in which llm_gray_band fallback may execute; score = 0.72 is reserved for the direct match threshold and is outside the gray band.
`cross_encoder_device_mode`	`cuda:0 deterministic when GPU is available; cpu deterministic fallback only when semantic_device_mode=cpu is configured`	Pins the execution device policy so primary scorer behavior is reproducible across deployments.

PARAMETER	VALUE	JUSTIFICATION
semantic_-calibration_-version	semantic-greeting-calibration-v1.0.0	Pins the calibration mapping from raw model scores to semantic probability estimates.

8.2 Deterministic semantic scorer registry

METHOD	ROLE / MODE	DETERMINIS-TIC	DESCRIPTION
cross_encoder	primary local	Yes	Primary local deterministic scorer using local://models/semantic/lsie-greeting-cross-encoder-v1 version lsie-greeting-cross-encoder-v1.0.0 with semantic-greeting-calibration-v1.0.0.
llm_gray_band	fallback azure_openai	Yes	Deterministic Azure OpenAI fallback invoked only when the calibrated cross_encoder score is within 0.58 <= score < 0.72 and gray-band fallback is enabled.
azure_llm_legacy	legacy azure_openai	Yes	Retired legacy direct Azure OpenAI semantic scorer retained only for historical attribution records.

8.3 Expected output schema

greeting_eval.schema.json JSON

```
{
 "type": "object",
 "required": [
  "reasoning",
  "is_match",
  "confidence_score"
 ],
 "properties": {
  "reasoning": {
   "type": "string",
   "enum": [
    "cross_encoder_high_match",
    "cross_encoder_high_nonmatch",
    "gray_band_llm_match",
    "gray_band_llm_nonmatch",
    "semantic_local_failure_fallback",
    "semantic_timeout",
    "semantic_error"
   ],
   "description": "Bounded semantic reason code; not a free-form rationale."
  },
  "is_match": {
   "type": "boolean"
  },
```

```
  "confidence_score": {
   "type": "number",
   "minimum": 0.0,
   "maximum": 1.0
  }
 },
 "additionalProperties": false
}
```

8.4 System prompt (canonical)

system_prompt.txt TEXT

```
You are a deterministic semantic evaluation fallback.
Your only task is to resolve ambiguous utterances that were routed into the gray band by the
↪  primary local semantic scorer.

Inputs:
1. Expected Greeting Rule
2. Actual Utterance

Evaluation constraints:
- Minor transcription errors and filler words are acceptable if the semantic intent remains
↪  unchanged.
- Major deviations in subject matter must be rejected.
- Do not produce free-form explanations.

Return strict JSON with exactly these fields:
1. reasoning: bounded reason code, one of ["gray_band_llm_match", "gray_band_llm_nonmatch"]
2. is_match: boolean
3. confidence_score: number between 0.00 and 1.00

Return the result strictly as valid JSON without markdown formatting.
```

SECTION 9

Desktop runtime topology

LSIE-MLF v4.0 runs as a Windows-native six-process desktop graph. Docker Compose, Redis, Celery, and local PostgreSQL are retired as normative desktop runtime dependencies. The launcher hydrates a python-build-standalone runtime with uv and starts child processes with spawn semantics and freeze_support.

9.1 Desktop process graph

PROCESS	ROLE	ALLOWED IMPORTS	STATE ACCESS
`ui_api_shell`	Hosts PySide6 operator console and localhost FastAPI API shell.	`PySide6, fastapi, uvicorn, pydantic`	Read-only SQLite through sqlite_reader with PRAGMA query_only=1.
`capture_supervisor`	Supervises scrcpy, ADB, and FFmpeg native subprocesses and emits capture/drift events.	`subprocess, pywin32, multiprocessing`	No direct SQLite writes; may read install manifest through parent-provided configuration.
`module_c_orchestrator`	Applies drift offsets, assembles 30-second segments, extracts AU12, attaches physiological context, captures bandit snapshot, and produces desktop IPC messages.	`mediapipe, numpy, av, multiprocessing.shared_memory`	Read-only experiment cache; no reward/posterior writes.
`gpu_ml_worker`	Consumes SharedMemory PCM windows and performs speech, semantic, acoustic, and ML inference.	`torch, ctranslate2, mediapipe, faster_whisper, sentence_transformers, transformers, parselmouth`	No direct SQLite writes.
`analytics_state_worker`	Sole SQLite writer. Computes reward, posterior updates, attribution, local metrics, and posterior deltas.	`sqlite3, pandas, scipy, pydantic, numpy`	Exclusive SQLite writer with WAL/NORMAL/busy_timeout/temp_store=MEMORY.
`cloud_sync_worker`	Synchronizes signed experiment bundles, sessions, telemetry, and posterior deltas with the cloud control plane.	`httpx, keyring, pydantic, cryptography`	Reads pending uploads and writes cloud sync acknowledgements through analytics_state_worker; stores secrets only in Windows Credential Manager.

9.2 Runtime base

PARAMETER	SPECIFICATION
Python distribution	python-build-standalone
Dependency manager	uv
Packaging shell	PyInstaller one-folder launcher shell only
Tier 1 Platform	Windows 11 x64

9.3 Desktop IPC contract

Control messages travel over multiprocessing.Queue. Thirty-second PCM windows travel through multiprocessing.shared_memory.SharedMemory.

Control Message Schema **JSON**

```
{
 "$schema": "http://json-schema.org/draft-07/schema#",
 "title": "DesktopIPCControlMessage",
 "description": "Queue envelope for v4.0 desktop IPC. Audio bytes live in SharedMemory, not
 ↪  JSON.",
 "type": "object",
 "required": [
  "message_type",
  "segment_id",
  "shared_memory_name",
  "byte_length",
  "checksum",
  "_au12_series",
  "_stimulus_time",
  "_active_arm",
  "_experiment_id",
  "_expected_greeting",
  "_x_max",
  "_bandit_decision_snapshot"
 ],
 "properties": {
  "message_type": {
   "type": "string",
   "enum": [
    "inference_handoff"
   ]
  },
  "segment_id": {
   "type": "string",
   "minLength": 16
  },
  "shared_memory_name": {
   "type": "string",
   "pattern": "^lsie_ipc_pcm_[0-9a-f-]+$"
  },
  "byte_length": {
   "type": "integer",
   "minimum": 1
  },
  "checksum": {
   "type": "string",
   "pattern": "^[0-9a-f]{64}$"
  },
  "_au12_series": {
   "type": "array",
   "items": {
    "type": "object",
    "required": [
     "timestamp_s",
     "intensity"
    ],
    "properties": {
```

```
      "timestamp_s": {
       "type": "number"
      },
      "intensity": {
       "type": "number",
       "minimum": 0.0,
       "maximum": 1.0
      }
     },
     "additionalProperties": false
    },
    "description": "Per-frame AU12 bounded intensity observations from the Orchestrator
    ↪  Container."
   },
   "_stimulus_time": {
    "type": [
     "number",
     "null"
    ],
    "description": "Drift-corrected UTC epoch seconds of stimulus injection, or null if no
    ↪  stimulus occurred in this segment."
   },
   "_active_arm": {
    "type": "string",
    "description": "Thompson Sampling arm identifier selected for this segment."
   },
   "_experiment_id": {
    "type": "integer",
    "description": "Persistent Store experiment row ID for this session."
   },
   "_expected_greeting": {
    "type": "string",
    "description": "Greeting rule text assigned to the active arm for semantic evaluation."
   },
   "_x_max": {
    "type": [
     "number",
     "null"
    ],
    "description": "Per-subject maximum raw D_mouth/IOD ratio observed during calibration.
    ↪  Carried as calibration telemetry; \u00a77B reward computation consumes bounded AU12
    ↪  intensities and does not divide P90 by this raw ratio."
   },
   "_bandit_decision_snapshot": {
    "$ref": "#/$defs/BanditDecisionSnapshot"
   },
   "_physiological_context": {
    "$ref": "#/$defs/PhysiologicalContext"
   }
  },
  "$defs": {
   "BanditDecisionSnapshot": {
    "$schema": "http://json-schema.org/draft-07/schema#",
    "title": "BanditDecisionSnapshot",
    "type": "object",
    "required": [
     "selection_method",
     "selection_time_utc",
     "experiment_id",
     "policy_version",
```

```
      "selected_arm_id",
      "candidate_arm_ids",
      "posterior_by_arm",
      "sampled_theta_by_arm",
      "expected_greeting",
      "decision_context_hash",
      "random_seed"
    ],
    "properties": {
      "selection_method": {
        "type": "string",
        "enum": [
          "thompson_sampling"
        ]
      },
      "selection_time_utc": {
        "type": "string",
        "format": "date-time"
      },
      "experiment_id": {
        "type": "integer"
      },
      "policy_version": {
        "type": "string"
      },
      "selected_arm_id": {
        "type": "string"
      },
      "candidate_arm_ids": {
        "type": "array",
        "items": {
          "type": "string"
        },
        "minItems": 1,
        "uniqueItems": true
      },
      "posterior_by_arm": {
        "type": "object",
        "minProperties": 1,
        "additionalProperties": {
          "$ref": "#/$defs/ArmPosterior"
        }
      },
      "sampled_theta_by_arm": {
        "type": "object",
        "additionalProperties": {
          "type": "number",
          "minimum": 0.0,
          "maximum": 1.0
        }
      },
      "expected_greeting": {
        "type": "string"
      },
      "decision_context_hash": {
        "type": "string",
        "pattern": "^[0-9a-f]{64}$",
        "description": "SHA-256 hash over the stable decision context used for deterministic
        ↪  offline arm selection and replay validation."
      },
      "random_seed": {
```

```
      "type": "integer",
      "minimum": 0,
      "maximum": 18446744073709551615,
      "description": "Unsigned 64-bit deterministic seed derived from SHA256(session_id ||
      ↪  segment_window_start_utc || stimulus_time || policy_version)."
    }
  },
  "$defs": {
    "ArmPosterior": {
      "type": "object",
      "required": [
        "alpha",
        "beta"
      ],
      "properties": {
        "alpha": {
          "type": "number",
          "exclusiveMinimum": 0.0
        },
        "beta": {
          "type": "number",
          "exclusiveMinimum": 0.0
        }
      },
      "additionalProperties": false
    }
  },
  "additionalProperties": false
},
"PhysiologicalContext": {
  "type": "object",
  "properties": {
    "streamer": {
      "oneOf": [
        {
          "type": "null"
        },
        {
          "$ref": "#/$defs/PhysiologicalSnapshot"
        }
      ]
    },
    "operator": {
      "oneOf": [
        {
          "type": "null"
        },
        {
          "$ref": "#/$defs/PhysiologicalSnapshot"
        }
      ]
    }
  },
  "additionalProperties": false,
  "anyOf": [
    {
      "required": [
        "streamer"
      ],
      "properties": {
        "streamer": {
```

```
        "$ref": "#/$defs/PhysiologicalSnapshot"
       }
      }
     },
     {
      "required": [
       "operator"
      ],
      "properties": {
       "operator": {
        "$ref": "#/$defs/PhysiologicalSnapshot"
       }
      }
     }
    ],
    "description": "Optional per-segment physiological context. When present, at least one of
    ↪  streamer or operator MUST contain a real PhysiologicalSnapshot; empty objects and
    ↪  null-only placeholders are invalid."
   }
  },
  "not": {
   "required": [
    "_audio_data"
   ]
  },
  "additionalProperties": false
 }
```

DIRTY STATE RECOVERY STEPS

1. At startup, unlink orphaned SharedMemory blocks matching lsie_ipc_*.
2. Checkpoint and truncate SQLite WAL when recovery policy requires it.
3. Reap stale scrcpy, ADB, and FFmpeg PIDs from the previous manifest.
4. Detect half-installed runtime environments before child processes spawn.
5. Drop incomplete pending-upload rows that fail schema validation before sync.

SECTION 10

Dependency and runtime matrix

10.1 System requirements

COMPONENT	REQUIRED VER	NOTES
Tier 1 host OS	`Windows 11 x64`	Production and developer Tier 1 host for v4.0.
Python runtime	`3.11.x`	Provided by python-build-standalone. Python 3.12+ is rejected because mediapipe == 0.10.9 has no supported PyPI wheel for this stack.

COMPONENT	REQUIRED VER	NOTES
Runtime resolver	`uv locked`	uv.lock is checked in and must remain consistent with pinned legacy requirements.
Production NVIDIA GPU	`Turing SM 7.5+`	Required for production GPU speech path with torch 2.4.x and ctranslate2 >= 4.5.0.
Developer Pascal override	`LSIE_DEV_FORCE_CPU_SPEECH=1`	Permits GTX 1080 Ti development by forcing speech transcription to CPU while leaving PyTorch semantic scoring and MediaPipe on GPU.
NVIDIA CUDA	`12.x`	Production CUDA runtime for torch and CTranslate2 stack.
cuDNN	`9.x`	Required by torch 2.4.x stack. v3.4 cuDNN 8/Pascal production justification is retired.
scrcpy	`3.3.4`	Required for raw Android USB audio/video extraction; v3.3.4 provides –no-playback and dual-instance port range support.
SQLite	`3.43+`	Local derived-state store in WAL mode. analytics_state_worker is the sole writer.
Windows Credential Manager	`OS-provided`	Secret storage backend accessed through keyring.

10.2 Python package pinning

PACKAGE	VERSION	CONTAINER(S)	PURPOSE
`faster-whisper`	`1.2.1`	gpu_ml_worker	Speech transcription wrapper using CTranslate2 runtime; desktop PCM arrives through SharedMemory.
`mediapipe`	`0.10.9`	module_c_-orchestrator, gpu_ml_worker	Face Mesh 478 landmark inference; legacy mp.solutions.face_mesh API preserved.
`praat-parselmouth`	`0.4.4`	worker	Praat acoustic feature extraction
`spacy`	`3.7.x`	worker	NLP preprocessing pipeline
`psycopg2-binary`	`2.9.x`	worker, api	PostgreSQL driver
`pandas`	`2.2.x`	analytics_state_worker	Time-series and analytics processing.
`scipy`	`1.13.*`	analytics_state_worker, gpu_ml_worker	Statistical computation and acoustic analytics.
`celery`	`5.4.x (v3.4 LTS only)`	legacy_v3_4	Legacy v3.4 distributed task queue; not normative for v4.0 desktop IPC.
`redis`	`5.x (v3.4 LTS only)`	legacy_v3_4	Legacy v3.4 Celery broker; not normative for v4.0 desktop IPC.
`fastapi`	`0.110.x`	ui_api_shell, cloud_api	Localhost desktop API shell and cloud API service framework.
`uvicorn`	`0.29.x`	ui_api_shell, cloud_api	ASGI server runtime for local and cloud FastAPI surfaces.

PACKAGE	VERSION	CONTAINER(S)	PURPOSE
pydantic	2.x	all_processes, cloud_api	Schema validation and schema-on-write privacy enforcement.
numpy	1.26.x	module_c_-orchestrator, gpu_ml_worker, analytics_state_worker	Vectorized AU12 calculations and ML preprocessing.
pycryptodome	⩾3.20.0	worker	AES-256-GCM encryption
patchright	1.58.2	worker	Stealth browser automation
TikTokLive	5.0.x	worker	TikTok WebSocket ingestion
PySide6	⩾6.11	ui_api_shell	Native Qt operator console hosted by the desktop UI/API shell.
torch	2.4.x	gpu_ml_worker	Local Cross-Encoder semantic scoring runtime and GPU tensor backend.
transformers	4.46.x	worker	Transformer runtime for deterministic local semantic scorer
sentence-transformers	3.2.x	worker	Cross-Encoder semantic similarity models
ctranslate2	⩾4.5.0	gpu_ml_worker	CTranslate2 runtime used by faster-whisper production GPU speech path with cuDNN 9-compatible stack.
pywin32	pinned in uv.lock	capture_-supervisor	Win32 Job Objects for native subprocess cleanup.
keyring	pinned in uv.lock	ui_api_shell, cloud_sync_worker	Windows Credential Manager backed secret storage.
httpx	pinned in uv.lock	desktop_-launcher, cloud_sync_worker	Runtime download, resumable first-run install, and HTTPS cloud sync.
cryptography	pinned in uv.lock	cloud_sync_worker, cloud_api	Ed25519 ExperimentBundle signature verification and cloud auth support.
sqlite-utils	optional	developer_tools	Developer inspection utilities only; runtime uses Python sqlite3.

SECTION 11

Variable extraction matrix

Each analytical variable produced within LSIE-MLF is traceable to its source module, data type, storage location, and extraction methodology.

VARIABLE PROFILE	EXTRACTION & ROUTING
Encoded Video Stream `byte stream (H.264 in MKV container)` Module A → Transient (kernel buffer)	scrcpy USB mirroring; H.264 encoded stream in MKV container via `/dev/bus/usb` passthrough.
Raw Audio PCM `byte stream (s16le)` Unit/range: 48 kHz mono Module A → Transient IPC pipe buffer	scrcpy raw audio capture; s16le 48 kHz mono to IPC Pipe.
Live Event Payloads `dict` Module B → Transient deque buffer	TikTokLive WebSocket ingestion; parsed protobuf event dictionaries.
Action Combo Trigger `bool` Unit/range: True/False Module B → Persistent Store events table	Composite temporal event correlation across concurrent chat actions.
UTC Drift Offset `float` Unit/range: seconds Module C → Transient variable	ADB epoch polling every 30s; `drift_offset` = host UTC − Android epoch.
Resampled Audio Chunks `byte stream (s16le)` Unit/range: 16 kHz mono Module C → Transient stdout pipe	FFmpeg resampling subprocess; 48 kHz → 16 kHz mono s16le on `pipe:1`.
3D Facial Landmarks `np.ndarray (478,3)` Unit/range: normalized coordinates Module C → Transient memory	MediaPipe Face Mesh in Module C; 478-vertex 3D landmark array per sampled frame.
AU12 Intensity Score `float` Unit/range: 0.0–5.0 (compute_intensity) or 0.0–1.0 (compute_bounded_intensity for Thompson Sampling reward pipeline) Module C → Persistent Store metrics table	AU12Normalizer (§7A) executed in Module C; IOD-normalized lip-corner distance with neutral baseline subtraction. Exposed as 0.0–5.0 (legacy diagnostic) and 0.0–1.0 (bounded, for reward pipeline).
Vocal Pitch F0 `float` Unit/range: 75–600 Hz Module D → Persistent Store metrics table	Legacy segment-level compatibility variable (deprecated as a canonical acoustic output in v3.3). Praat via parselmouth; fundamental frequency extraction from speech audio. Downstream consumers must migrate to the windowed §7D F0 fields.

VARIABLE PROFILE	EXTRACTION & ROUTING
Jitter `float` Unit/range: 0–1 ratio Module D → Persistent Store metrics table	Legacy segment-level compatibility variable (deprecated as a canonical acoustic output in v3.3). Praat via parselmouth; cycle-to-cycle pitch perturbation ratio. Downstream consumers must migrate to the windowed §7D jitter fields.
Shimmer `float` Unit/range: 0–1 ratio Module D → Persistent Store metrics table	Legacy segment-level compatibility variable (deprecated as a canonical acoustic output in v3.3). Praat via parselmouth; cycle-to-cycle amplitude perturbation ratio. Downstream consumers must migrate to the windowed §7D shimmer fields.
ASR Transcription `string` Unit/range: UTF-8 text Module D → Persistent Store transcripts table	faster-whisper with CTranslate2 INT8 backend; segment-level speech-to-text.
Semantic Match `JSON object` Unit/range: confidence 0–1 Module D → Persistent Store evaluation table	Deterministic semantic scorer (§8); primary local Cross-Encoder with optional deterministic llm_gray_band fallback. Output contract contains bounded `reasoning` reason code, `is_match`, and `confidence_score`.
Evaluation Variance `float` Unit/range: 0–infinity Module E → Persistent Store experiments table	Thompson Sampling posterior statistics; variance of Beta distribution per arm.
External Context Metadata `JSON dict` Module F → Persistent Store context table	patchright headless browser scraping; creator and trending topic metadata.
gated_reward `float` Unit/range: [0.0, 1.0] Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); r_t = `P90` × `semantic_gate`.
p90_intensity `float` Unit/range: [0.0, 1.0] Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); 90th percentile of bounded AU12 in stimulus window.
semantic_gate `int` Unit/range: {0, 1} Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); binary gate from `is_match`: 1 if True, 0 otherwise.

VARIABLE PROFILE	EXTRACTION & ROUTING
n_frames_in_window `int` Unit/range: [0, +infinity) Module E → Phase3_EncounterLog JSON payload	`compute_reward()` (§7B); count of AU12 observations in stimulus window.
au12_baseline_pre `float | null` Unit/range: [0.0, +infinity) or null Module E → Phase3_EncounterLog JSON payload and Persistent Store attribution_score table	`compute_reward()` (§7B) and §7E attribution diagnostics; mean bounded AU12 in the pre-stimulus window, or null.
stimulus_time `float` Unit/range: UTC epoch seconds Module C → Transient variable	Drift-corrected UTC epoch of stimulus injection; defines baseline and measurement windows for `compute_reward()` (§7B).
RMSSD (Streamer) `float | null` Unit/range: milliseconds, >= 0 or null Module C → Persistent Store physiology_log table	Derived at segment assembly from normalized physiological chunks buffered in Module C (§4.B.2, §4.C.4). Uses a trailing 5-minute window with ibi precedence over session. Forced to null when `validity_ratio < PHYSIO_VALIDITY_MIN`.
RMSSD (Operator) `float | null` Unit/range: milliseconds, >= 0 or null Module C → Persistent Store physiology_log table	Derived at segment assembly from normalized physiological chunks buffered in Module C (§4.B.2, §4.C.4). Uses a trailing 5-minute window with ibi precedence over session. Forced to null when `validity_ratio < PHYSIO_VALIDITY_MIN`.
Heart Rate (Streamer/Operator) `int | null` Unit/range: 20–300 BPM or null Module C → Persistent Store physiology_log table	Rounded mean of overlapping `heart_rate_items_bpm` values in the trailing derivation window (§4.C.4); null when no heart-rate values exist.
Physiological Validity Ratio `float` Unit/range: [0.0, 1.0] Module C → Persistent Store physiology_log table	`validity_ratio = sum(valid_sample_count) / max(sum(expected_sample_count), 1)` over the trailing derivation window (§4.C.4).
Physiological Validity Flag `bool` Unit/range: True / False Module C → Persistent Store physiology_log table	`is_valid = (validity_ratio ⩾ PHYSIO_VALIDITY_MIN)`; default threshold 0.80 (§4.C.4).
Physiological Source Kind `string` Unit/range: {'ibi', 'session'} Module C → Persistent Store physiology_log table	ibi when the snapshot is derived from interbeat-interval chunks; otherwise session when derived from provider-session HRV chunks (§4.C.4).

VARIABLE PROFILE	EXTRACTION & ROUTING
Physiological Derivation Method `string` Unit/range: {'provider', 'server'} Module C → Persistent Store physiology_log table	`server` for IBI-derived RMSSD; `provider` for provider-session HRV snapshots unless later recomputed (§4.C.4).
Physiological Window Length `int` Unit/range: seconds, > 0 Module C → Persistent Store physiology_log table	Length of the trailing derivation window; default `PHYSIO_DERIVE_WINDOW_S = 300` (§4.C.4).
Physiological Freshness `float` Unit/range: seconds, >= 0 Module C → Persistent Store physiology_log table	Computed at segment assembly: `wall_clock - source_timestamp_utc` (§4.C.4).
Physiological Staleness Flag `bool` Unit/range: True / False Module C → Persistent Store physiology_log table	`freshness_s > PHYSIO_STALENESS_THRESHOLD_S` (default 600s) (§4.C.4).
Co-Modulation Index `float | null` Unit/range: [-1.0, +1.0] or null Module E → Persistent Store co-modulation_log table	Pearson correlation on 1-minute resampled, time-aligned valid streamer/operator RMSSD sequences over a 10-minute rolling window (§7C).
Physiological Coverage Ratio `float` Unit/range: [0.0, 1.0] Module E → Persistent Store co-modulation_log table	Fraction of 1-minute analysis bins with valid paired RMSSD observations over the 10-minute co-modulation window (§7C).
F0 Validity (Measure Window) `bool` Unit/range: True / False Module D → Persistent Store metrics table	Observational acoustic validity gating (§7D); true only when W_measure satisfies voiced-coverage and stable-island thresholds.
F0 Validity (Baseline Window) `bool` Unit/range: True / False Module D → Persistent Store metrics table	Observational acoustic validity gating (§7D); true only when W_baseline satisfies voiced-coverage and stable-island thresholds.
Perturbation Validity (Measure Window) `bool` Unit/range: True / False Module D → Persistent Store metrics table	Perturbation-span validity gating (§7D); true only when W_measure contains an eligible perturbation span.

VARIABLE PROFILE	EXTRACTION & ROUTING
Perturbation Validity (Baseline Window) `bool` Unit/range: True / False Module D → Persistent Store metrics table	Perturbation-span validity gating (§7D); true only when W_baseline contains an eligible perturbation span.
Voiced Coverage (Measure Window) `float` Unit/range: >= 0 seconds Module D → Persistent Store metrics table	Cumulative voiced coverage in W_measure computed under §7D voiced-only masking.
Voiced Coverage (Baseline Window) `float` Unit/range: >= 0 seconds Module D → Persistent Store metrics table	Cumulative voiced coverage in W_baseline computed under §7D voiced-only masking.
F0 Mean (Measure Window) `float | null` Unit/range: 75–600 Hz or null Module D → Persistent Store metrics table	Mean voiced-only F0 in W_measure under §7D; null when f0_valid_measure is false.
F0 Mean (Baseline Window) `float | null` Unit/range: 75–600 Hz or null Module D → Persistent Store metrics table	Mean voiced-only F0 in W_baseline under §7D; null when f0_valid_baseline is false.
F0 Delta (Semitones) `float | null` Unit/range: real-valued semitone difference or null Module D → Persistent Store metrics table	12 * log2(f0_mean_measure_hz / f0_mean_baseline_hz) under §7D; null when either dependent F0 mean is null or baseline F0 is non-positive.
Jitter Mean (Measure Window) `float | null` Unit/range: 0–1 ratio or null Module D → Persistent Store metrics table	Praat local jitter computed on the selected perturbation span in W_measure under §7D; null when perturbation_valid_measure is false.
Jitter Mean (Baseline Window) `float | null` Unit/range: 0–1 ratio or null Module D → Persistent Store metrics table	Praat local jitter computed on the selected perturbation span in W_baseline under §7D; null when perturbation_valid_baseline is false.

VARIABLE PROFILE	EXTRACTION & ROUTING
Jitter Delta `float | null` Unit/range: real-valued difference or null Module D → Persistent Store metrics table	jitter_mean_measure - jitter_mean_baseline under §7D; null when either dependent jitter mean is null.
Shimmer Mean (Measure Window) `float | null` Unit/range: 0–1 ratio or null Module D → Persistent Store metrics table	Praat local shimmer computed on the selected perturbation span in W_measure under §7D; null when perturbation_valid_measure is false.
Shimmer Mean (Baseline Window) `float | null` Unit/range: 0–1 ratio or null Module D → Persistent Store metrics table	Praat local shimmer computed on the selected perturbation span in W_baseline under §7D; null when perturbation_valid_baseline is false.
Shimmer Delta `float | null` Unit/range: real-valued difference or null Module D → Persistent Store metrics table	shimmer_mean_measure - shimmer_mean_baseline under §7D; null when either dependent shimmer mean is null.
Segment ID `string` Unit/range: 64-character lowercase hex Module C → InferenceHandoffPayload and Persistent Store attribution_event table	Deterministic SHA-256 over stable segment identity fields (`session_id`, `segment_window_start_utc`, `segment_window_end_utc`) in Module C (§6.1, §7E).
Bandit Decision Snapshot `JSON object` Module C → InferenceHandoffPayload and Persistent Store attribution_event table	Captured in Module C immediately after Thompson Sampling arm selection and before any posterior update; includes selected arm, candidate arms, posterior parameters, sampled thetas when available, and expected greeting.
semantic_p_match `float` Unit/range: [0.0, 1.0] Module D → Persistent Store evaluation table and attribution_score table	Deterministic semantic probability estimate produced by the active §8 semantic method; primary local Cross-Encoder in v3.4, optional gray-band fallback.
semantic_reason_code `string` Unit/range: bounded enum Module D → Persistent Store evaluation table	Bounded semantic route/result code emitted by the deterministic semantic scorer contract (§8).

VARIABLE PROFILE	EXTRACTION & ROUTING
soft_reward_candidate `float | null` Unit/range: [0.0, 1.0] or null Module E → Persistent Store attribution_score table	`soft_reward_candidate = semantic_p_match * proxy_reward` under §7E observational attribution only.
au12_lift_p90 `float | null` Unit/range: [0.0, 1.0] or null Module E → Persistent Store attribution_score table	`max(0, P90_post - au12_baseline_pre)` under §7E.
au12_lift_peak `float | null` Unit/range: [0.0, 1.0] or null Module E → Persistent Store attribution_score table	`max(0, Pmax_post - au12_baseline_pre)` under §7E.
au12_peak_latency_ms `float | null` Unit/range: milliseconds or null Module E → Persistent Store attribution_score table	Latency from stimulus timestamp to the peak AU12 observation in the post-stimulus measurement window under §7E.
sync_peak_corr `float | null` Unit/range: [-1.0, 1.0] or null Module E → Persistent Store attribution_score table	Maximum lagged Pearson correlation over the configured lag range under §7E lag-aware synchrony analytics.
sync_peak_lag `int | null` Unit/range: lag units or null Module E → Persistent Store attribution_score table	Lag value associated with `sync_peak_corr` under §7E lag-aware synchrony analytics.
outcome_link_lag_s `float` Unit/range: seconds, >= 0 Module E → Persistent Store event_outcome_link table	Deterministic event→outcome lag in seconds stored on EventOutcomeLink records under §7E.
attribution_finality `string` Unit/range: {'online_provisional', 'offline_final'} Module E → Persistent Store attribution tables	Lifecycle state assigned by Module E attribution finalization jobs under §7E.
SharedMemory Audio Segment Name `string` Unit/range: lsie_ipc_pcm_* Module C → Queue control message only	Generated by module_c_orchestrator when staging a 30-second PCM window in SharedMemory.

VARIABLE PROFILE	EXTRACTION & ROUTING
SharedMemory Byte Length `integer` Unit: bytes Range: >0 Module C → Queue control message only	Measured from the exact PCM byte window staged for gpu_ml_worker.
SharedMemory Checksum `hex string` Range: SHA-256 Module C → Queue control message and diagnostics	SHA-256 over staged PCM bytes before SharedMemory handoff.
Bandit Random Seed `unsigned integer` Range: 0..18446744073709551615 Module C → BanditDecisionSnapshot	First 8 bytes of SHA-256(session_id || segment_window_start_utc || stimulus_time || policy_version), interpreted big-endian.
Posterior Delta `object` Range: 0≤delta≤1 Module E → SQLite posterior_delta_log and cloud PostgreSQL	analytics_state_worker computes fractional Beta-Bernoulli update and emits idempotent delta.
Cloud Event ID `UUIDv5 string` Unit/range: UUID Module F → SQLite pending_uploads and cloud posterior_delta_log	UUIDv5 over client_id, segment_id, experiment_id, and arm_id for exactly-once sync.

SECTION 12

Error handling and recovery specification

LSIE-MLF v4.0 defines deterministic recovery behavior over the desktop process graph. Container restarts and broker retries are replaced by child-process supervision, Queue backpressure, SharedMemory cleanup, SQLite WAL recovery, and cloud-sync backlog handling.

12.1 Hardware device loss

MODULE	BEHAVIOR
capture_supervisor	USB reconnection polling every 2s for 60s; Win32 Job Object terminates stale scrcpy/ADB/FFmpeg children before revival.

12.2 Worker process crash

MODULE	BEHAVIOR
capture_supervisor	Parent process reaps native children through Job Object and emits deterministic capture failure event.

12.3 Temporal drift fault

MODULE	BEHAVIOR
module_c_-orchestrator	ADB drift freezes after 3 failures and resets after 300 seconds; corrected_ts = original_ts + last_valid_offset while frozen.

12.4 SharedMemory leak

MODULE	BEHAVIOR
module_c_-orchestrator	Dirty State Recovery unlinks orphaned lsie_ipc_* blocks at startup; owner zeroizes and unlinks after gpu_ml_worker acknowledgement.

12.5 ML import failure

MODULE	BEHAVIOR
gpu_ml_worker	Preflight fails process readiness if torch/ctranslate2/mediapipe/faster_whisper cannot co-load on a production Turing-or-newer GPU.

12.6 Pascal speech incompatibility

MODULE	BEHAVIOR
gpu_ml_worker	Developer machines may set LSIE_DEV_FORCE_CPU_SPEECH=1; production must use Turing SM 7.5+ GPU speech path.

12.7 Acoustic data quality

MODULE	BEHAVIOR
gpu_ml_worker	Insufficient voiced coverage, invalid perturbation spans, or Praat extraction failure produces deterministic false validity flags and null dependent acoustic outputs.

12.8 SQLite Busy/Crash recovery

MODULE	BEHAVIOR
analytics_state_-worker	Single writer uses WAL, synchronous=NORMAL, busy_timeout=5000, temp_-store=MEMORY; restart verifies schema, checkpoints/truncates WAL if required, and resumes pending uploads.

12.9 Reward bounds fault

MODULE	BEHAVIOR
analytics_state_-worker	Rewards outside [0,1] are rejected before posterior update; Gate 0 replay verifies r_t = P90 × G_t and fractional Beta-Bernoulli update.

12.10 Network disconnection

MODULE	BEHAVIOR
cloud_sync_worker	Enter offline mode; local inference and posterior updates continue; pending_uploads retry with backoff.

12.11 Cloud duplicate

MODULE	BEHAVIOR
cloud_sync_worker	Posterior deltas use UNIQUE(segment_id, client_id, arm_id) and INSERT … ON CONFLICT DO NOTHING; cloud posterior UPDATE occurs only on successful insert.

12.12 Privacy perimeter violation

MODULE	BEHAVIOR
cloud API	extra=forbid schemas reject raw PCM, decoded frames, voiceprints, raw provider bodies, complete physiological chunks, and free-form semantic rationales.

12.13 Operator UI exit

MODULE	BEHAVIOR
ui_api_shell	Parent shutdown closes child process graph and Job Objects; local SQLite remains consistent through WAL recovery.

SECTION 13

Autonomous implementation audit checklist

This checklist defines mandatory v4.0 implementation controls. A workstream may not merge until the relevant Gate 0 and runtime-topology controls pass.

#	AUDIT ITEM	VERIFICATION CRITERION
1	Runtime topology	Exactly six v4.0 desktop processes are defined: ui_api_shell, capture_supervisor, module_c_orchestrator, gpu_ml_worker, analytics_state_worker, cloud_sync_worker.
2	Import discipline	Only gpu_ml_worker imports the complete heavy ML set torch, ctranslate2, mediapipe, and faster_whisper; module_c_orchestrator may import MediaPipe only for AU12/Face Mesh.
3	Embedded runtime	python-build-standalone and uv.lock hydrate the desktop runtime; PyInstaller is used only for the launcher shell.
4	Windows subprocess supervision	capture_supervisor reaps ADB, scrcpy, and FFmpeg through Win32 Job Objects.
5	SharedMemory IPC	30-second PCM windows travel through SharedMemory; Queue control messages contain shared_memory_name, byte_length, and checksum, not _audio_data.
6	SharedMemory cleanup	Consumers attach with track=False; Dirty State Recovery unlinks lsie_ipc_* orphaned blocks; audio blocks are zeroized before unlink.
7	Gate 0 corpus	Gate 0 replay corpus exists and blocks all workstreams.
8	Deterministic segment identity	segment_id = SHA256(session_id || canonical window_start || canonical window_end) and 30-second segment windows are preserved.
9	ADB drift contract	ADB drift freezes after 3 failures and resets after 300 seconds.
10	Reward pipeline	Reward pipeline implements §7B unchanged: P90 aggregation, binary semantic gate, fractional Beta-Bernoulli posterior updates with Beta(1,1) prior, and stimulus window boundaries [t+0.5s, t+5.0s].
11	Bandit snapshot determinism	BanditDecisionSnapshot.random_seed and decision_context_hash are required and captured before posterior update.
12	SQLite local state	SQLite uses WAL/NORMAL/busy_timeout/temp_store=MEMORY and analytics_state_worker is the sole writer.
13	Privacy perimeter	No raw media, hydrated provider bodies, complete physiological chunks, voiceprints, or free-form semantic rationales persist locally or in cloud.
14	Cloud OAuth	Cloud sync uses OAuth 2.0 Authorization Code with PKCE; refresh tokens are stored only in Windows Credential Manager via keyring.
15	Signed ExperimentBundle	Experiment bundles are Ed25519-signed and cached locally only after signature verification.
16	Posterior delta exactly-once	Posterior deltas are idempotent through UNIQUE(segment_id, client_id, arm_id) and INSERT … ON CONFLICT DO NOTHING.
17	Production hardware floor	Production GPU path is validated on Turing SM 7.5+ with CUDA 12.x and cuDNN 9.x.

#	AUDIT ITEM	VERIFICATION CRITERION
18	Pascal developer override	Pascal development requires LSIE_DEV_FORCE_CPU_SPEECH=1 for CPU speech transcription while preserving other GPU paths.
19	macOS Tier 2 deferral	macOS App Nap, Keychain, process-group, Hardened Runtime, and MPS routing are documented as Tier 2 v4.1 design intent only.

— END OF SPECIFICATION —